

How it works:

Create features from time series:

- Lag features: Value from t-1, t-2, t-7 days ago

- Rolling statistics: 7-day average, 30-day std dev

- Date features: Day of week, month, is_weekend, is_holiday

- Domain features: Promotions, weather, events

Train regression model:

- XGBoost, Random Forest, Linear Regression

- Predict: $y(t) = f(\text{lag1}, \text{lag7}, \text{day_of_week}, \text{rolling_mean}, \dots)$

Example:

```
python

# Feature engineering for sales forecasting
df['lag_1'] = df['sales'].shift(1) # Yesterday's sales
df['lag_7'] = df['sales'].shift(7) # Last week same day
df['rolling_mean_7'] = df['sales'].rolling(7).mean() # 7-day average
df['day_of_week'] = df['date'].dt.dayofweek
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

# Train XGBoost
X = df[['lag_1', 'lag_7', 'rolling_mean_7', 'day_of_week', 'is_weekend']]
y = df['sales']
model = XGBRegressor()
model.fit(X, y)
```

Advantages:

- Can capture non-linear patterns
- Easy to add external features (weather, promotions)
- Flexible - use any regression algorithm
- Can handle multiple targets

When to Use:

- Complex patterns that ARIMA can't capture
- Many external factors influence target
- Non-linear relationships
- Want feature importance interpretability

Dataset Example:

Sales Forecasting Dataset (Kaggle)

- **Link:** <https://www.kaggle.com/datasets/rohitsahoo/sales-forecasting>
- **Contents:** Historical sales data with dates
- **Use case:** Practice building forecasting models
- **Typical columns:** Date, Store, Item, Sales, Promotions

Evaluation Metrics:

For Forecasting:

- **MAE (Mean Absolute Error):** Average absolute difference
- **RMSE (Root Mean Squared Error):** Penalizes large errors more
- **MAPE (Mean Absolute Percentage Error):** Percentage error
- **R²:** How much variance is explained

Example:

Actual: [100, 105, 98, 110, 95]
Predicted: [102, 103, 100, 108, 97]

$$\begin{aligned} \text{MAE} &= |100-102| + |105-103| + |98-100| + |110-108| + |95-97| / 5 \\ &= (2 + 2 + 2 + 2 + 2) / 5 = 2.0 \end{aligned}$$

$$\text{MAPE} = (|100-102|/100 + \dots) / 5 \times 100\% = 2.0\%$$

Key Takeaways:

- Time series has trend, seasonality, and noise
- ARIMA: Classical, good for short-term, requires stationarity
- Prophet: Modern, easy to use, great for business data
- ML regressors: Flexible, good for complex patterns
- Always validate on future data (never past)

Slide 16: Anomaly Detection

Task Definition:

Anomaly Detection identifies unusual patterns or outliers in data that don't conform to expected behavior.

Also called: Outlier detection, novelty detection, fraud detection

Real-World Examples:

Example 1: Fraud Detection

Scenario: Credit card transaction monitoring

Normal behavior:

- Purchases in home city
- Typical spending amounts (\$20-200)
- Usual merchants (grocery, gas, restaurants)
- Daytime transactions

Anomalous behavior (red flags):

- \$5,000 purchase in foreign country
- Multiple transactions within minutes
- Unusual merchant category (luxury goods when user typically buys groceries)
- Transaction at 3 AM

Detection approach:

Real-time transaction:
Amount: \$8,000
Location: Another country
Time: 2 AM
Merchant: Electronics

Anomaly score: 0.95 (very unusual!)
→ Block transaction
→ Send alert to customer
→ Require verification

Business impact:

- Prevent fraudulent charges

- Protect customers
- Reduce chargebacks
- Save millions in fraud losses

Example 2: System Failures

Scenario: Data center monitoring

Normal behavior:

- Server CPU: 40-60%
- Memory usage: 50-70%
- Disk I/O: 10-30 MB/s
- Network latency: 5-15 ms

Anomalous behavior (potential failure):

- CPU suddenly spikes to 95%
- Memory leak: gradual increase to 98%
- Disk writes drop to 0 (disk failure)
- Network latency jumps to 500ms

Detection approach:

Continuous monitoring:

CPU usage pattern deviates from normal

Memory trend shows abnormal growth

Disk I/O drops unexpectedly

Anomaly detected:

→ Alert DevOps team

→ Trigger automated failover

→ Log incident for analysis

Business impact:

- Prevent system downtime
- Early warning before complete failure
- Faster incident response
- Maintain SLA (Service Level Agreement)

Other Examples:

- **Manufacturing:** Defect detection before product leaves factory
- **Healthcare:** Abnormal vital signs, disease outbreak detection
- **Cybersecurity:** Intrusion detection, unusual network traffic
- **E-commerce:** Fake reviews, bot activity
- **Energy:** Equipment malfunction prediction

Techniques:

1. Statistical Thresholds

Approach: Define normal range, flag values outside it

Simple Methods:

A. Z-Score (Standard Deviation)

$Z\text{-score} = (\text{value} - \text{mean}) / \text{standard_deviation}$

Normal range: $|Z\text{-score}| < 3$

Anomaly: $|Z\text{-score}| \geq 3$

Example:

Data: [50, 52, 48, 51, 49, 150, 47]

Mean = 63.9, Std = 37.9

Value 150:

$Z\text{-score} = (150 - 63.9) / 37.9 = 2.27$

Not quite anomaly (< 3)

But value 150 is clearly unusual!

Limitation: Assumes normal distribution

B. IQR (Interquartile Range)

Q1 = 25th percentile

Q3 = 75th percentile

$IQR = Q3 - Q1$

Lower bound = $Q1 - 1.5 \times IQR$

Upper bound = $Q3 + 1.5 \times IQR$

Anything outside bounds = anomaly

Example:

Data: [10, 12, 11, 13, 10, 15, 14, 100]

Q1 = 10.5, Q3 = 14.5, IQR = 4

Upper bound = $14.5 + 1.5 \times 4 = 20.5$

Value 100 $> 20.5 \rightarrow$ Anomaly!

C. Moving Average

Calculate rolling average and standard deviation

Flag points that deviate significantly

Example:

Sales normally $\$5000 \pm \500 per day

Today: \$15,000

Deviation: $(\$15,000 - \$5,000) / \$500 = 20$ std devs!

Clear anomaly

Advantages:

- Simple to implement and understand
- Fast computation
- No training needed
- Interpretable

Limitations:

- Assumes statistical distribution
- Fixed thresholds may not adapt
- Doesn't capture complex patterns
- Many false positives with noisy data

2. ML-Based Detection

Approach: Train models to learn normal patterns, flag deviations

A. Isolation Forest

How it works:

- Builds random decision trees
- Anomalies are easier to isolate (fewer splits needed)
- Normal points require more splits to isolate

Intuition:

Imagine points in 2D space:
- 100 points clustered together (normal)
- 1 point far away (anomaly)

To isolate the outlier: 1-2 splits
To isolate a normal point: 10+ splits

Anomaly score = How easily point is isolated

Example:

```
python

from sklearn.ensemble import IsolationForest

# Training data (normal behavior)
X_train = [[50], [52], [48], [51], [49], [53]]

# Fit model
clf = IsolationForest(contamination=0.1) # Expect 10% anomalies
clf.fit(X_train)

# Predict
X_test = [[51], [150], [48]] # New data
predictions = clf.predict(X_test)

# Output: [1, -1, 1]
# 1 = normal, -1 = anomaly
# 150 is flagged as anomaly
```

B. Autoencoders (Deep Learning)

How it works:

- Neural network trained to reconstruct normal data
- High reconstruction error = anomaly

Architecture:

Input → Encoder (compress) → Bottleneck → Decoder (reconstruct) → Output

Training: Only on normal data
Normal input → Low reconstruction error
Anomalous input → High reconstruction error (model hasn't seen this pattern)

Example:

Normal transaction: [50, 1, 0, 0] (amount, foreign, night, luxury)
Reconstructed: [49.8, 0.9, 0.1, 0.1]
Error: Low → Normal

Fraudulent: [5000, 1, 1, 1]
Reconstructed: [52, 0.9, 0.2, 0.1] (model tries to make it "normal")
Error: Very high → Anomaly!

C. One-Class SVM

How it works:

- Learn boundary around normal data

- Points outside boundary = anomalies

Use case:

- When you have lots of normal examples
- Few or no anomaly examples
- Need to define "normal" region

D. LSTM for Time Series Anomalies

How it works:

- LSTM predicts next value based on sequence
- Large prediction error = anomaly

Example:

Time series: [10, 12, 11, 13, 12, ...]
LSTM learns pattern

Next value:
LSTM predicts: 12.5
Actual: 13.2 → Small error, normal
Actual: 100 → Huge error, anomaly!

Evaluation Metrics:

Confusion Matrix:

	Predicted	
	Anomaly	Normal
Actual		
Anomaly	TP	FN
Normal	FP	TN

TP = True Positive (correctly detected anomaly)
FP = False Positive (normal flagged as anomaly)
FN = False Negative (missed anomaly)
TN = True Negative (correctly identified normal)

Key Metrics:

- **Precision:** Of flagged anomalies, how many are real? $TP / (TP + FP)$
- **Recall:** Of real anomalies, how many did we catch? $TP / (TP + FN)$
- **F1-Score:** Harmonic mean of precision and recall

Trade-offs:

High threshold → Few alerts → High precision, low recall (miss anomalies)
Low threshold → Many alerts → Low precision, high recall (false alarms)

Goal: Balance based on business cost

- Missing fraud: Lose money
- False alarms: Annoy customers

Find optimal threshold based on costs

Real-World Considerations:

1. Class Imbalance:

- Anomalies are rare (0.1-1% of data)
- Models may predict "normal" for everything and still be 99% accurate
- Need specialized techniques

2. Concept Drift:

- Normal behavior changes over time
- Model needs retraining
- Example: COVID changed "normal" shopping behavior

3. Alert Fatigue:

- Too many false alarms → Ignored alerts
- Need high precision
- Balance sensitivity vs specificity

4. Real-time Requirements:

- Fraud detection: <100ms latency
- Can't use complex models
- Need fast, simple algorithms

Key Takeaways:

- Anomaly detection identifies unusual patterns
- Statistical methods: Fast, simple, interpretable
- ML methods: Better accuracy, handle complex patterns
- Isolation Forest is popular choice
- Always validate with domain experts
- Balance false positives vs false negatives based on business costs

Slide 17: Time Series Industry Use Cases

Industries:

1. Finance (Risk & Fraud)

A. Risk Management

Stock Price Forecasting:

- Predict future stock movements
- Portfolio optimization
- Trading strategy development
- Models: ARIMA, LSTM, Transformer-based

Example:

Historical data: Daily stock prices for 5 years

Features:

- Technical indicators (moving averages, RSI, MACD)
- Volume, volatility
- Market indices
- News sentiment

Forecast: Next day's closing price ± confidence interval

Credit Risk Scoring:

- Predict loan default probability over time
- Monitor changing risk levels
- Dynamic credit limits

Market Volatility Prediction:

- VIX forecasting
- Risk assessment for portfolios
- Derivatives pricing

B. Fraud Detection

Transaction Monitoring:

Normal pattern:

User typically spends \$20-100

Transactions in home city

Business hours

Consistent merchants

Anomaly triggers:

\$5,000 transaction (10x normal)

Foreign country

3 AM

New merchant type

Action: Block + alert + require verification

Account Takeover Detection:

- Login from unusual location
- Unusual behavior after login
- Rapid account changes

Money Laundering Detection:

- Unusual transaction patterns
- Structuring (multiple small transactions)
- Circular money flows

Business Impact:

- **JP Morgan:** Saves \$200M+ annually from fraud detection
- **Typical ROI:** 5-10x investment in fraud prevention systems
- **Industry standard:** Block 95%+ of fraud with <1% false positive rate

2. Retail (Demand Forecasting)

A. Sales Forecasting

Why it matters:

- Optimize inventory (not too much, not too little)
- Staff scheduling
- Promotional planning
- Supply chain coordination

Example: Grocery Chain

Forecast daily sales for each product by store:

Historical data:

- 3 years of sales data
- Promotions and discounts
- Holidays and events
- Weather data
- Local demographics

Model: Prophet + XGBoost ensemble

Features:

- Lag sales (yesterday, last week, last year)
- Day of week, month, season
- Is_holiday, is_payday
- Promotion (yes/no, discount %)
- Temperature, precipitation
- Local events (sports, concerts)

Output:

Product A, Store 1: 150 units ± 15 (confidence interval)

Product B, Store 1: 75 units ± 8

Business actions:

- Restock Product A: 160 units (buffer for uncertainty)
- Promotions on slow-moving items
- Staff 5 cashiers (predicted busy day)

Accuracy impact:

5% forecast error → 8% stockouts OR 12% excess inventory

2% forecast error → 3% stockouts OR 5% excess inventory

Better forecasting:

- Reduce stockouts by 50% = More sales
- Reduce excess inventory by 60% = Lower costs
- Net impact: 3-5% margin improvement

B. Dynamic Pricing

Use case: Adjust prices based on demand forecast

High demand forecast → Increase prices slightly

Low demand forecast → Promotions, discounts

Example: Uber surge pricing

Forecast: High ride demand (Friday 11 PM, rainy)

Action: Surge multiplier 1.8x

Result: More drivers online, supply meets demand

C. Inventory Optimization

Safety stock calculation:

Forecast demand: 1000 units/week ± 100

Lead time: 2 weeks

Safety stock = Z-score × std_dev × √lead_time

= 1.96 × 100 × √2

= 277 units

Order point = (forecast × lead_time) + safety_stock

= (1000 × 2) + 277

= 2,277 units

When inventory drops to 2,277: Reorder

Business Impact:

- **Walmart:** Reduced inventory by 30% with better forecasting
- **Amazon:** Anticipatory shipping (move inventory before orders)
- **Typical improvement:** 20-40% inventory reduction, 10-25% stockout reduction

3. Operations (Predictive Maintenance)

A. Equipment Failure Prediction

Scenario: Manufacturing plant with expensive machinery

Traditional approach:

- Scheduled maintenance (every X hours/days)
- Problem: Maintain too early (wasted cost) or too late (breakdown)

Predictive maintenance approach:

Sensor data:

- Vibration levels
- Temperature
- Power consumption
- Oil pressure
- Acoustic signature

Time series anomaly detection:

Normal vibration: 0.5-1.5 mm/s

Current: 2.8 mm/s and rising → Anomaly!

Forecast:

Failure probability increasing

Estimated time to failure: 5-7 days

Action:

Schedule maintenance in 3 days

Order spare parts

Avoid unplanned downtime

B. Predictive Maintenance for Fleets

Example: Airline engines

Monitor 1000+ sensors per engine in real-time

Predict: Remaining useful life (RUL)

Engine A:

Current condition: Good

Predicted RUL: 2000 flight hours

Next maintenance: Scheduled

Engine B:

Anomaly detected: Bearing temperature rising

Predicted RUL: 100 flight hours (risk!)

Action: Inspect at next landing, replace proactively

Business Impact:

- **Unplanned downtime cost:** \$10,000-\$1M per hour (industry-dependent)
- **Predictive maintenance savings:** 20-30% maintenance costs
- **Typical improvement:** 70% reduction in breakdowns, 25% reduction in maintenance costs

C. Energy Demand Forecasting

Use case: Electrical grid management

Forecast electricity demand:

- Hourly for next 24 hours
- Daily for next week
- Monthly for planning

Factors:

- Historical consumption patterns
- Weather forecasts (temperature drives AC/heating)
- Day of week, holidays
- Economic activity

Output:

Peak demand: 12-2 PM, 6500 MW

Low demand: 3-5 AM, 2800 MW

Actions:

- Activate additional generators during peak
- Schedule maintenance during low-demand periods
- Balance load across grid

Cross-Industry Benefits:

Common ROI:

- **Forecast accuracy improvement:** 20-50% over baseline methods
- **Cost savings:** 5-20% through optimization
- **Revenue increase:** 2-10% from better inventory/pricing
- **Risk reduction:** 30-70% fewer failures/frauds

Key Success Factors:

1. Good quality historical data
2. Domain expertise (understand the business)
3. Regular model updates (adapt to changes)
4. Integration with operations (forecasts must drive actions)
5. Monitoring and feedback loops

PART 4: REGRESSION & CLASSIFICATION IN PRACTICE

Slide 18: Regression in Industry

When to Use Regression:

Predicting Continuous Numerical Values:

- Prices
- Revenue
- Demand
- Temperature
- Risk scores (continuous 0-100)
- Customer lifetime value

Real-World Examples:

Example 1: House Price Prediction

Business Context:

- Real estate platform needs accurate pricing

- Help sellers set realistic prices
- Help buyers understand fair market value
- Enable mortgage lenders to assess property value

Input Features:

Property Characteristics:

- Square footage
- Number of bedrooms
- Number of bathrooms
- Age of house
- Lot size

Location:

- Neighborhood
- School district quality
- Distance to city center
- Crime rate

Market Factors:

- Recent sales in area
- Time on market (seasonality)
- Economic indicators

Model Approach:

Algorithm: XGBoost or Random Forest

Why: Captures non-linear relationships (e.g., location premium)

Feature Engineering:

- Price per square foot in neighborhood
- Ratio: bedrooms to total rooms
- Age × condition interaction
- Distance to amenities (schools, transit, shops)

Training:

- Historical sales: 100,000 properties
- Train/test split: 80/20
- Cross-validation: 5-fold

Output:

Predicted price: \$425,000

Confidence interval: \$405,000 - \$445,000

Evaluation:

MAE: \$18,000 (average error)

RMSE: \$25,000 (penalizes large errors)

R²: 0.87 (explains 87% of price variation)

Business translation:

- 87% accuracy in pricing
- Within \$25K of actual price (typical 5-6% error)
- Competitive advantage for listing platform

Example 2: Diamond Price Prediction

Business Context:

- E-commerce jewelry site needs dynamic pricing
- Assess value of diamonds for purchase/resale
- Customer wants price estimate before visiting store

Input Features (The 4 C's + more):

Intrinsic:

- Carat (weight)
- Cut (quality: Ideal, Premium, Good, Fair, Poor)
- Color (D-Z scale, D is best)
- Clarity (FL, IF, VVS1, VVS2, VS1, VS2, SI1, SI2, I1, I2, I3)

Physical:

- Depth %
- Table %
- Dimensions (x, y, z)

Certificate:

- GIA, AGS, IGI (certification authority)

Model Approach:

Algorithm: Gradient Boosting (XGBoost/LightGBM)

Feature Engineering:

- Carat²: Non-linear price increase with size
- Cut × Clarity: Interaction effect
- Volume = x × y × z
- Premium_cut: Binary (Ideal/Premium vs others)

Log-transform target: log(price)
Why: Price growth is exponential with carat

Training:
Dataset: 50,000 diamonds with prices
Validation: Hold-out test set

Output:
log(price) prediction → exp() → \$5,450
Confidence: ± \$200

Business Value:

Automated Pricing:

- Instant quotes for customers
- Dynamic inventory pricing
- Competitive market positioning

Risk Management:

- Don't overpay when purchasing
- Don't underprice when selling
- Identify mispriced opportunities

Customer Trust:

- Transparent pricing
- "Fair price guarantee"
- Price matching confidence

Other Regression Applications:

Revenue Forecasting:

- Monthly/quarterly revenue predictions
- Budget planning
- Investor reporting

Demand Prediction:

- Units to manufacture
- Inventory levels
- Staffing requirements

Customer Lifetime Value:

- Predict total revenue from customer
- Marketing spend optimization
- Segment high-value customers

Energy Consumption:

- Predict kWh usage
- Load balancing
- Cost estimation

Salary Prediction:

- HR compensation planning
- Job market analysis
- Candidate expectations

Slide 19: Classification in Industry

When to Use Classification:

Predicting Categories or Binary Outcomes:

- Yes/No decisions
- Multiple categories
- Risk levels (Low/Medium/High)
- Customer segments

Real-World Examples:

Example 1: Fraud Detection

Business Context:

- Financial institution processes millions of transactions daily
- Need to identify fraudulent transactions in real-time
- Balance: Catch fraud vs don't block legitimate transactions

Problem Formulation:

- **Type:** Binary classification
- **Classes:** Fraud (1) vs Legitimate (0)
- **Challenge:** Highly imbalanced (0.1% are fraud)

Input Features:

Transaction Attributes:

- Amount
- Merchant category
- Transaction type (online, in-store, ATM)
- Time of day, day of week

User Behavior:

- Time since last transaction
- Frequency of transactions (velocity)
- Average transaction amount (user history)
- Location (distance from home)

Device/Channel:

- Device ID
- IP address location
- Browser/app version
- Card present vs not present

Historical Patterns:

- Transactions in last 24h, 7d, 30d
- Number of unique merchants
- Cross-border transactions (yes/no)

Model Approach:

Algorithm: XGBoost or LightGBM

Why:

- Handles imbalanced data well
- Fast inference (<10ms requirement)
- Feature importance for investigations

Handling Imbalance:

- Use scale_pos_weight parameter
- SMOTE oversampling
- Focus on F1-score, not accuracy

Training:

- 10M historical transactions
- 10,000 known frauds
- Time-based split (train on past, test on future)

Threshold Tuning:

- Default: probability > 0.5 → Fraud
- Adjusted: probability > 0.3 → Fraud (catch more, but more false positives)
- Balance based on cost: Missing fraud (\$500 avg) vs Declining legit (\$50 customer service + reputation)

Evaluation:

Confusion Matrix:

Actual	Predicted		
	Fraud	Legit	
Fraud	85	15	(Recall: 85%)
Legit	200	9,700	(Precision: 30%)

Metrics:

- Precision: $85 / (85 + 200) = 30\%$ (of flagged transactions, 30% are fraud)
- Recall: $85 / (85 + 15) = 85\%$ (catch 85% of frauds)
- F1-Score: 44%

Business Impact:

- Catch 85% of frauds = Prevent \$425K losses per 10K transactions
- 200 false positives = \$10K in customer service + some reputation cost
- Net benefit: \$415K

Compared to no model:

- All frauds succeed: \$500K losses
- AI saves \$415K (83% reduction in fraud losses)

Production Considerations:

Real-time Requirements:

- Latency: <50ms
- Throughput: 10,000 transactions/second peak
- Uptime: 99.99%

Monitoring:

- Track precision and recall daily
- Alert if fraud rate changes significantly
- Retrain monthly with new fraud patterns

Fallback:

- If model fails: Use rule-based system
- Never block all transactions (business continuity)

Example 2: Customer Churn Prediction

Business Context:

- Subscription service (SaaS, telecom, streaming)
- Acquiring new customers is 5-7x more expensive than retaining
- Want to identify customers likely to cancel and intervene proactively

Problem Formulation:

- **Type:** Binary classification
- **Classes:** Churn (1) vs Retain (0)
- **Prediction horizon:** Next 30 days

Input Features:

Usage Behavior:

- Logins per week (decreasing = risk)
- Features used (active vs inactive)
- Time spent on platform
- Last activity date

Engagement:

- Support tickets (many = frustrated?)
- Feature adoption rate
- Response to emails
- Community participation

Demographics:

- Account age
- Contract type (month-to-month vs annual)
- Payment history (missed payments)
- Company size (B2B)

Financial:

- MRR (Monthly Recurring Revenue)
- Discounts received
- Payment method
- Upgrade/downgrade history

Model Approach:

Algorithm: Logistic Regression or Random Forest

Why:

- Interpretable (can explain to business teams)
- Probability scores (prioritize high-risk customers)
- Feature importance (understand drivers)

Training:

- 50,000 customers
- 6 months historical data
- 5,000 churned (10% churn rate)

Feature Engineering:

- Trend features: (logins_last_7d - logins_prev_7d)
- Engagement score: Weighted combination of activities
- Days_since_last_login
- Contract_expiring_soon: Binary (next 30 days)

Validation:

- Time-based: Train on Jan-Jun, test on Jul
- Ensures model works on future data

Evaluation:

Confusion Matrix:

	Predicted		
	Churn	Retain	
Actual			
Churn	750	250	(Recall: 75%)
Retain	500	8,500	(Precision: 60%)

Metrics:

- Precision: $750 / (750 + 500) = 60\%$
- Recall: $750 / (750 + 250) = 75\%$
- F1-Score: 67%

Business Translation:

- Identify 750 of 1000 at-risk customers
- Intervene: Offer discount, personal outreach, feature training

Intervention Cost:

- \$50 per customer (outreach + discount)
- $750 \text{ customers} \times \$50 = \$37,500$

Retention Impact:

- Without intervention: $750 \text{ churn} \times \$100 \text{ MRR} \times 12 \text{ months} = \900K lost revenue
- With intervention: Save 50% = \$450K retained revenue
- Net benefit: $\$450\text{K} - \$37.5\text{K} = \$412.5\text{K}$

ROI: 11x return on intervention investment

Intervention Strategy:

Churn Risk Tiers:

- High risk (>70% probability): Personal call from account manager
- Medium risk (40-70%): Automated email with discount offer
- Low risk (<40%): No intervention, continue monitoring

Timing:

- Act 2-4 weeks before predicted churn
- Too early: Unnecessary cost
- Too late: Already decided to leave

Other Classification Applications:

Lead Scoring:

- Classify leads: Hot/Warm/Cold
- Prioritize sales team effort
- Improve conversion rates

Email Routing:

- Classify support tickets by topic
- Route to appropriate team
- Reduce response time

Medical Diagnosis:

- Disease presence (yes/no)
- Disease type (multi-class)
- Assist doctors, not replace

Credit Approval:

- Approve/Deny loan applications
- Classify risk level
- Regulatory compliance required

Slide 20: Feature Selection & Data Prep

Key Steps:

Most ML gains come from proper data preparation, not fancy algorithms!

1. Remove Leakage

What is Leakage? Using information in features that wouldn't be available at prediction time, causing artificially high performance.

Example 1: Target Leakage

Problem: Predict if customer will buy product

Bad feature: "purchase_confirmation_sent"

Why bad: This is only known AFTER purchase

Leakage: Model learns if confirmation sent → purchase happened (100% accurate but useless)

Fix: Remove feature, only use pre-purchase information

Example 2: Temporal Leakage

Problem: Predict tomorrow's stock price

Bad: Train on entire dataset including future data

Why bad: Model sees future prices during training

Fix: Time-based split

- Train: 2020-2023
- Test: 2024
- Never let model see future when predicting past

Example 3: Data Leakage in Medical Prediction

Problem: Predict if patient has disease

Bad feature: "prescribed_medication_X"

Why bad: Medication is prescribed AFTER diagnosis

Model learns: medication → disease (circular logic)

Fix: Only use symptoms, test results available before diagnosis

How to Detect Leakage:

- Model is "too good" (>95% accuracy on difficult problem)
- Features perfectly correlated with target
- Model performs terribly in production vs training
- Feature importance shows suspicious features at top

2. Handle Missing Values

Strategy depends on why data is missing:

A. Missing Completely at Random (MCAR)

Example: Sensor failed randomly

Strategy:

- Delete rows (if <5% missing)
- Impute with mean/median
- Impute with forward fill (time series)

B. Missing at Random (MAR)

Example: Survey respondents skip income question more when income is low

Strategy:

- Create "is_missing" indicator feature
- Impute based on other features
- Use model-based imputation

C. Missing Not at Random (MNAR)

Example: Sick patients don't take all medical tests

Strategy:

- Missing is informative!
- Create "is_missing" binary feature
- Impute carefully (missing might mean high risk)

Common Imputation Methods:

```
python

# Mean imputation (numeric)
df['age'].fillna(df['age'].mean())

# Median (robust to outliers)
df['income'].fillna(df['income'].median())

# Mode (categorical)
df['category'].fillna(df['category'].mode()[0])

# Forward fill (time series)
df['temperature'].fillna(method='ffill')

# Model-based imputation
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5)
df_imputed = imputer.fit_transform(df)
```

3. Encode Categorical Variables

A. Label Encoding (Ordinal)

Use when: Categories have meaningful order

Example: Education level
Low: 0, Medium: 1, High: 2

```
Python:
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['education_encoded'] = le.fit_transform(df['education'])
```

B. One-Hot Encoding (Nominal)

Use when: No order, just categories

Example: Color
Red: [1, 0, 0]
Blue: [0, 1, 0]
Green: [0, 0, 1]

```
Python:
df_encoded = pd.get_dummies(df, columns=['color'])
```

Result:

color_Red	color_Blue	color_Green
1	0	0
0	1	0

C. Target Encoding (Advanced)

Encode category by its relationship to target

Example: Predict house price by neighborhood
Neighborhood A: avg price \$300K → encode as 300000
Neighborhood B: avg price \$500K → encode as 500000

Advantage: Captures relationship with target
Warning: Can cause overfitting, use with care (CV)

When you have many categories (>50):

Options:

1. Group rare categories: "Other"

2. Use embeddings (deep learning)

3. Target encoding

4. Leave-one-out encoding

4. Scale Numeric Features

Why scaling matters:

- Some algorithms sensitive to feature scales (KNN, SVM, Neural Networks)
- Tree-based models (XGBoost, Random Forest) don't need scaling

A. Standardization (Z-score)

Transform to: mean=0, std=1

Formula: $(x - \text{mean}) / \text{std}$

Example:
Age: [25, 30, 35, 40, 45]
Mean: 35, Std: 7.07
Scaled: [-1.41, -0.71, 0, 0.71, 1.41]

Python:
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df['age_scaled'] = scaler.fit_transform(df[['age']])

B. Min-Max Scaling

Transform to: range [0, 1]

Formula: $(x - \text{min}) / (\text{max} - \text{min})$

Example:
Age: [25, 30, 35, 40, 45]
Min: 25, Max: 45
Scaled: [0, 0.25, 0.5, 0.75, 1.0]

Python:
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df['age_scaled'] = scaler.fit_transform(df[['age']])

C. Robust Scaling

Use median and IQR (robust to outliers)

Formula: $(x - \text{median}) / \text{IQR}$

Good when: Data has outliers

Python:

```
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
df['income_scaled'] = scaler.fit_transform(df[['income']])
```

Important: Fit on train, transform on test!

```
python

# WRONG: Fit on entire dataset (causes leakage)
scaler.fit_transform(df)

# CORRECT: Fit on train only
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test) # Use train statistics
```

Key Takeaway:

"Garbage in, garbage out"

Most ML improvements come from:

- Better features (80%)
- Better algorithms (20%)

Spend time on data preparation! It's not glamorous but it's where real impact happens.

Slide 21: Model Evaluation in Practice

Regression Metrics:

1. MAE (Mean Absolute Error)

Formula: Average of $|\text{actual} - \text{predicted}|$

Example:

Actual: [100, 150, 200]

Predicted: [110, 140, 190]

Errors: [10, 10, 10]

$\text{MAE} = (10 + 10 + 10) / 3 = 10$

Interpretation: On average, predictions are off by 10 units

Advantage: Easy to understand, same unit as target

Use when: All errors equally bad

2. RMSE (Root Mean Squared Error)

Formula: $\sqrt{\text{average of (actual - predicted)}^2}$

Example:

Actual: [100, 150, 200]

Predicted: [110, 120, 190]

Errors²: [100, 900, 100]

$MSE = (100 + 900 + 100) / 3 = 366.67$

$RMSE = \sqrt{366.67} = 19.1$

Interpretation: Penalizes large errors more

Advantage: Penalizes outliers (large errors hurt more)

Use when: Large errors are particularly bad

3. R² (R-squared / Coefficient of Determination)

Formula: $1 - (SS_{\text{residual}} / SS_{\text{total}})$

Range: 0 to 1 (higher is better)

- $R^2 = 1$: Perfect predictions

- $R^2 = 0.8$: Model explains 80% of variance

- $R^2 = 0$: Model no better than predicting mean

Example:

Actual prices: [100K, 200K, 300K, 400K]

Mean: 250K

Baseline (always predict mean):

Errors: [150K, 50K, 50K, 150K]

$SS_{\text{total}} = 150^2 + 50^2 + 50^2 + 150^2 = 50,000$

Your model predictions: [110K, 190K, 310K, 390K]

Errors: [10K, 10K, 10K, 10K]

$SS_{\text{residual}} = 10^2 + 10^2 + 10^2 + 10^2 = 400$

$R^2 = 1 - (400 / 50,000) = 0.992$ (99.2% variance explained)

Interpretation: Model is excellent!

Which metric to use?

MAE: When you want intuitive, interpretable error

RMSE: When large errors are costly

R²: When you want to know % variance explained

Best practice: Report all three

Classification Metrics:

The Confusion Matrix:

	Predicted	
	Positive	Negative
Actual		
Positive	TP	FN
Negative	FP	TN

TP = True Positive (correctly identified positive)

FP = False Positive (incorrectly labeled as positive)

FN = False Negative (missed positive)

TN = True Negative (correctly identified negative)

1. Precision

Formula: $TP / (TP + FP)$

Question: Of all predicted positives, how many are actually positive?

Example: Spam detection

Predicted spam: 100 emails

Actually spam: 90

Precision = $90/100 = 0.90$ (90%)

Interpretation: When model says "spam", it's right 90% of the time

Use when: False positives are costly

- Medical screening (don't want false alarms)
- Fraud detection (don't block legitimate customers)

2. Recall (Sensitivity)

Formula: $TP / (TP + FN)$

Question: Of all actual positives, how many did we catch?

Example: Disease detection

Total diseased: 100 patients

Detected: 85

Recall = $85/100 = 0.85$ (85%)

Interpretation: We catch 85% of disease cases

Use when: False negatives are costly

- Medical diagnosis (don't miss sick patients)
- Fraud (catch fraudsters even if some false alarms)

3. F1-Score

Formula: $2 \times (Precision \times Recall) / (Precision + Recall)$

Harmonic mean of precision and recall

Example:

Precision = 0.90

Recall = 0.85

$F1 = 2 \times (0.90 \times 0.85) / (0.90 + 0.85) = 0.874$

Use when: Need balance between precision and recall

Trade-off Example:

Fraud Detection:

High Threshold (0.8):

- Precision: 90% (few false alarms)
- Recall: 60% (miss some frauds)
- Good when: False alarms very costly

Low Threshold (0.3):

- Precision: 50% (many false alarms)
- Recall: 95% (catch almost all frauds)
- Good when: Missing fraud very costly

Balance:

- Threshold 0.5
- Precision: 70%
- Recall: 80%
- F1: 75%

Metrics Must Align with Business Impact:

Example 1: Medical Screening

Goal: Don't miss any cancer cases

Priority: High Recall (catch all cancers)
Accept: Lower Precision (more false alarms acceptable)

Reason: Missing cancer = patient dies (unacceptable)
False alarm = More tests (inconvenient but safe)

Metric focus: Recall > 95%

Example 2: Email Spam

Goal: Don't block important emails

Priority: High Precision (spam folder has only spam)
Accept: Lower Recall (some spam in inbox okay)

Reason: Blocking important email = lost business
Some spam in inbox = minor annoyance

Metric focus: Precision > 98%

Example 3: Recommendation System

Goal: Show relevant products

Metric: Precision@K (of top 10 recommendations, how many clicked?)
Why: User only sees top 10, don't care about rank 100

Business metric: Click-through rate, revenue per user
ML metric must correlate with business metric!

Key Principle:

Choose evaluation metrics based on business costs, not just statistical properties.

Always ask:

- What's the cost of false positive?
- What's the cost of false negative?
- Which error is more expensive?
- What metric actually matters to the business?

Slide 22: Common Real-World Pitfalls

Pitfall 1: Using Wrong AI Domain

Mistake: Applying inappropriate AI technique to problem

Example:

Problem: Predict customer churn (will they cancel?)
Data: Tabular (customer age, usage, support tickets, etc.)

WRONG: Use Computer Vision (why would you?!)
WRONG: Use LLM (overkill, expensive, slow)
RIGHT: Use Classical ML (XGBoost on structured data)

How to avoid:

1. Identify data type first (images? text? tables?)
2. Match problem to appropriate domain

- 3. Start simple, add complexity only if needed

Pitfall 2: Overfitting

Mistake: Model memorizes training data, fails on new data

Example:

Training set: 95% accuracy
Test set: 65% accuracy

→ Model has overfit! Learned noise, not patterns

Causes:

- Too complex model for amount of data
- No regularization
- Training too long
- Data leakage

How to avoid:

- Use cross-validation
- Regularization (L1, L2)
- Early stopping
- More training data
- Simpler model

Pitfall 3: Ignoring Cost and Latency

Mistake: Building accurate model that's too slow or expensive for production

Example:

Built: BERT-based text classifier
Accuracy: 94% (great!)
Latency: 2 seconds per prediction
Cost: \$10,000/month at expected volume

Problem: Need <100ms for web app, budget is \$1,000/month

Result: Model never deployed, wasted 3 months

How to avoid:

- Define constraints upfront (latency, budget)
- Calculate costs before building
- Prototype with simple approach first
- Benchmark performance early

Pitfall 4: Choosing LLMs Unnecessarily

Mistake: Using LLM when simpler approach works

Example:

Task: Classify product reviews as positive/negative

Option A: GPT-4 API

- Cost: \$0.03 per review
- Latency: 2 seconds
- 1M reviews/month = \$30,000

Option B: TF-IDF + Logistic Regression

- Cost: \$100/month (server)
- Latency: 5 milliseconds
- Accuracy: 89% (vs GPT-4: 91%)

Decision: Option B saves \$29,900/month for 2% accuracy drop

How to avoid:

- Always try simple baseline first
- Calculate cost at expected scale
- Use LLMs only when necessary
- Consider hybrid (LLM for complex, rules for simple)

PART 5: LARGE LANGUAGE MODELS (LLMs)

Slide 23: What Are LLMs?

Definition:

Large Language Models (LLMs) are deep neural networks with billions of parameters, trained on massive amounts of text data to understand and generate human-like language.

Characteristics:

"Large":

- Billions of parameters (GPT-3: 175B, GPT-4: estimated 1.7T)
- Require massive compute to train (\$10M-\$100M training costs)
- Result: Emergent capabilities not seen in smaller models

"Language":

- Trained on text from internet, books, code
- Learn grammar, facts, reasoning, style
- Understand context, nuance, idioms

"Models":

- Based on Transformer architecture
- Self-attention mechanism
- Pre-trained, then fine-tuned or prompted

Popular Examples:

GPT (Generative Pre-trained Transformer) - OpenAI:

- GPT-3, GPT-3.5, GPT-4
- General purpose, versatile
- API-based access

LLaMA (Large Language Model Meta AI) - Meta:

- Open-source weights
- Various sizes (7B to 70B parameters)
- Community-driven fine-tunes

Claude - Anthropic:

- Focus on safety and helpfulness
- Long context window
- Constitutional AI training

Other Notable:

- PaLM (Google)
- Gemini (Google)
- Mistral (Mistral AI)
- Llama 3 (Meta, latest)

Reference:

Hugging Face NLP Course

- **Link:** <https://huggingface.co/learn/nlp-course>
- Comprehensive free course on NLP and LLMs
- Practical tutorials with code
- Covers transformers, fine-tuning, deployment

Slide 24: LLM Use Cases in Industry

Real-World Applications:

1. Chatbots / Conversational AI

- Customer support automation
- Virtual assistants
- Internal knowledge bots
- Lead qualification

2. Document Summarization

- Summarize long reports
- Meeting notes to action items
- Legal document summaries
- Research paper abstracts

3. Code Assistance

- Code completion (GitHub Copilot)
- Bug detection and fixing
- Code explanation and documentation
- Test generation

4. Content Generation

- Marketing copy
- Product descriptions
- Email templates

- Social media posts

5. Data Extraction

- Extract structured data from unstructured text
- Form filling automation
- Invoice/receipt parsing
- Resume information extraction

6. Translation and Localization

- Multi-language support
- Cultural adaptation
- Technical documentation translation

The Reality:

LLMs Are Powerful:

- ✓ Handle diverse tasks with single model
- ✓ Few-shot learning (learn from examples)
- ✓ Natural language interface
- ✓ Impressive capabilities

But LLMs Are Costly:

- ✗ Expensive inference (\$0.01-0.10 per request)
- ✗ High latency (1-5 seconds)
- ✗ Require significant infrastructure if self-hosted
- ✗ Energy intensive

Key Decision: Use LLMs when their flexibility and capability justify the cost. For focused, well-defined tasks, classical ML is often better.

Slide 25: When to Train or Fine-Tune a Model

Train or Fine-Tune When:

1. Domain Language is Specific

Example: Legal contracts, medical records

Problem: General LLM doesn't understand domain terminology
"Indemnification clause" (legal)
"Myocardial infarction" (medical)

Solution: Fine-tune on domain-specific text
Result: Model understands specialized language

2. Off-the-Shelf Models Underperform

Example: Customer support for specific product

General LLM: Generic responses, hallucinations
Fine-tuned model: Accurate product knowledge, consistent tone

Benchmark:
GPT-4 (zero-shot): 70% accuracy
Fine-tuned on company data: 90% accuracy

3. Enough Labeled Data Exists

Fine-tuning requirements:

- Minimum: 100-1000 examples
- Good: 10,000+ examples
- Excellent: 100,000+ examples

More data = better fine-tuned performance

4. Cost Savings at Scale

Scenario: 1M requests/month

GPT-4 API: \$30,000/month
Fine-tuned smaller model (self-hosted): \$5,000/month

Break-even: ~50K requests/month
Above that: Self-hosting cheaper

Avoid Training When:

1. Data is Limited

< 100 examples: Don't fine-tune (will overfit)
Use: Few-shot prompting with examples in prompt

2. Task is Generic

Examples:

- General text summarization
- Simple Q&A
- Translation

General LLMs already excel at these
No need for custom training

3. Need Flexibility

Requirements change frequently
New use cases emerge
Multiple different tasks

Solution: Use general LLM with prompt engineering
Easier to adapt than retraining

4. Lack of ML Expertise

Fine-tuning requires:

- ML engineering skills
- Infrastructure setup
- Monitoring and maintenance

If don't have team: Use API services

Decision Framework:

Is domain language highly specialized?
→ Yes → Consider fine-tuning

Is volume >50K requests/month?
→ Yes → Economics favor self-hosting

Do you have >1000 labeled examples?
→ Yes → Fine-tuning viable

Do you have ML engineering team?
→ Yes → Can support fine-tuned model

All Yes → Fine-tune
Otherwise → Use API + prompt engineering

Slide 26: End-to-End Case Study Exercise

Exercise: Design Complete AI System

Choose ONE of these scenarios and design an end-to-end solution:

Option 1: Invoice Processing System

Business Context: Company receives 1000 invoices/day via email. Currently processed manually (5 minutes each).

Your Tasks:

1. **Identify domain:** Which AI domain(s)?
2. **Choose techniques:** What specific methods?
3. **Define metrics:** How measure success?
4. **Sketch architecture:** High-level system design

Considerations:

- What accuracy is needed?
- What latency is acceptable?
- What's the budget?
- How handle errors?

Option 2: Customer Support Chatbot

Business Context: E-commerce company gets 5000 support tickets/day. Want to automate 70% of common questions.

Your Tasks:

1. **Identify domain:** Which AI domain(s)?
2. **Choose techniques:** Intent classification? RAG? LLM?
3. **Define metrics:** How measure success?
4. **Sketch architecture:** High-level system design

Considerations:

- Which questions to automate vs human?
- How handle escalation?
- What response time needed?
- Cost per conversation?

Option 3: Sales Forecasting System

Business Context: Retail chain with 100 stores. Need accurate weekly sales forecasts per store per product.

Your Tasks:

1. **Identify domain:** Which AI domain(s)?
2. **Choose techniques:** ARIMA? Prophet? ML regressors?
3. **Define metrics:** How measure success?
4. **Sketch architecture:** High-level system design

Considerations:

- What features matter (seasonality, promotions, weather)?
- How often retrain?
- What forecast horizon?
- How integrate with inventory system?

Exercise Format:

Document Structure:

1. Problem Analysis

- Business goal:

- Current state:

- Target state:

- Constraints:

2. AI Domain Selection

- Primary domain:

- Why this domain:

- Alternatives considered:

3. Technical Approach

- Data sources:

- Models/algorithms:

- Why chosen:

4. Metrics & Evaluation

- Success metrics:

- Target performance:

- Business impact:

5. Architecture Sketch

[Simple diagram showing:]

- Data flow

- Components

- Integrations

- Monitoring

6. Implementation Plan

- Phase 1 (MVP):

- Phase 2 (Production):

- Phase 3 (Scale):

7. Risks & Mitigation

- Technical risks:

- Business risks:

- Mitigation strategies:

Evaluation Criteria:

Your solution will be evaluated on:

1. **Problem understanding:** Do you grasp the business need?
2. **Domain selection:** Right AI approach?
3. **Technical feasibility:** Can this actually work?
4. **Cost consideration:** Is it economically viable?
5. **Completeness:** Did you think through details?
6. **Practicality:** Would this work in real world?

Slide 27: Hands-On Assessment

Assessment Task:

You must complete all four steps:

Step 1: Pick One Industry Problem

Choose from:

- **Healthcare:** Patient readmission prediction
- **Finance:** Credit risk assessment
- **Retail:** Product recommendation
- **Manufacturing:** Predictive maintenance
- **E-commerce:** Dynamic pricing
- **Telecom:** Network anomaly detection
- **Transportation:** Delivery time estimation
- **Agriculture:** Crop yield prediction

Or propose your own (get approval first)

Step 2: Map to AI Domain

Question to answer:

Primary Domain: [Computer Vision / NLP / Time Series / Tabular ML / LLM]

Why this domain:

- Data type:
- Problem type:
- Business constraints:

Secondary domains (if hybrid approach):

- Domain 2:
- How it integrates:

Example:

Problem: Patient readmission prediction

Primary Domain: Tabular ML (Classification)

Why:

- Data: Structured patient records (age, diagnoses, medications, previous visits)
- Problem: Binary classification (readmit within 30 days: yes/no)
- Constraints: Need explainability for doctors, <1 second prediction time

Secondary Domain: NLP (optional)

Why: Could incorporate clinical notes for richer features

Integration: NLP extracts features from notes → combine with structured data

Step 3: Propose Solution

Components to specify:

A. Data Requirements:

- What data needed:
- How much data (volume):
- Data quality requirements:
- How to collect/access:

B. Model Selection:

- Algorithm choice:
- Why not alternatives:
- Expected performance:
- Training requirements:

C. Features:

- Input features (list top 10):
- Feature engineering needed:
- How handle missing values:

D. Deployment:

- Batch vs real-time:
- Latency requirements:
- Throughput requirements:
- Infrastructure needs:

E. Monitoring:

- What to monitor:
- Alert conditions:
- Retraining frequency:

Step 4: Justify Decisions

Answer these questions:

1. Why this approach over alternatives?

Approach A (your choice): [Explain]

vs

Approach B: [Why not]

vs

Approach C: [Why not]

2. How handle edge cases?

Edge case 1:

Edge case 2:

Edge case 3:

3. What could go wrong?

Technical risks:

Business risks:

Mitigation:

4. Expected ROI?

Costs:

- Development:
- Operation:
- Maintenance:

Total annual cost:

Benefits:

- Time savings:
- Error reduction:
- Revenue increase:

Total annual benefit:

ROI: [Benefit / Cost]

Payback period: [Months]

Deliverable:

Format: 3-5 page document or 10-15 slide presentation

Sections:

1. Executive Summary (1 paragraph)

- 2. Problem Statement (1/2 page)
- 3. Proposed Solution (2 pages)
- 4. Technical Details (1 page)
- 5. ROI Analysis (1/2 page)
- 6. Risks & Mitigation (1/2 page)

Assessment Rubric:

Criteria	Weight	Points
Problem Understanding	20%	/20
Domain Selection	20%	/20
Technical Soundness	25%	/25
Justification & Reasoning	20%	/20
Practicality & ROI	15%	/15
TOTAL	100%	/100

Grading:

- 90-100: Excellent - Production-ready thinking
- 80-89: Good - Minor gaps, mostly solid
- 70-79: Satisfactory - Has understanding, needs refinement
- <70: Needs improvement - Fundamental gaps

Slide 28: Key Outcomes of Day 10

By the End of Day 10, You Should:

1. Know Where Each AI Domain Fits

You can answer:

"I have images of defective parts"
→ Computer Vision, Object Detection or Image Classification
"I have customer complaints in text"
→ NLP, Text Classification + Sentiment Analysis
"I have historical sales data with timestamps"
→ Time Series Forecasting
"I have customer attributes in spreadsheet"
→ Tabular ML, Classification or Regression
"I need to answer complex questions from documents"
→ NLP + LLM (RAG system)

Mental Model:

Problem → Data Type → AI Domain → Technique → Model

2. Map Problems to Correct Techniques

You can do technical scoping:

Given: Business problem You produce:

- Data requirements
- Model approach
- Architecture sketch

- Cost estimate
- Timeline estimate
- Risk assessment

Example:

Problem: "We want to reduce customer churn"

Your analysis:

- Domain: Tabular ML (Classification)
- Data needed: Usage logs, support tickets, payment history
- Model: XGBoost or Logistic Regression
- Features: Engagement metrics, payment issues, support interactions
- Metrics: Precision-Recall (cost of false positives vs false negatives)
- Architecture: Batch prediction weekly, export high-risk list to CRM
- Cost: \$2K setup, \$500/month operation
- Timeline: 4-6 weeks to production
- Risk: Data quality, integration with CRM, intervention effectiveness

3. Speak Confidently About Use Cases

You can communicate effectively with:

A. Business Stakeholders:

- Explain what AI can and cannot do
- Set realistic expectations
- Discuss ROI and timeline
- Address concerns about accuracy, cost, privacy

B. Technical Teams:

- Discuss model choices and trade-offs
- Explain architecture decisions
- Review technical feasibility
- Plan implementation details

C. Interviews:

- Answer case study questions
- Propose solutions to business problems
- Justify technical choices
- Demonstrate end-to-end thinking

4. Be Ready for Real Projects

You have:

- ✓ Framework for analyzing AI problems
- ✓ Knowledge of major AI domains and techniques
- ✓ Understanding of trade-offs (accuracy, cost, latency, complexity)
- ✓ Ability to design end-to-end systems
- ✓ Awareness of common pitfalls and how to avoid them

You can:

- ✓ Scope AI projects
- ✓ Choose appropriate techniques
- ✓ Estimate costs and timelines
- ✓ Design architectures

- ✓ Define success metrics
- ✓ Identify risks and plan mitigation

Next Steps:

- Apply these frameworks to your own projects
 - Practice with more case studies
 - Stay updated on new techniques and tools
 - Build portfolio of AI solutions
 - Prepare for interviews with this knowledge
-

Slide 29: What Comes Next

Next Phases Will Focus On:

1. Real Project Builds

Hands-on implementation:

- Build complete AI systems from scratch
- Work with real datasets
- Deploy to production-like environments
- Handle data pipeline, model training, deployment, monitoring

Project ideas:

- Image classification web app
- Chatbot with RAG
- Time series forecasting dashboard
- Recommendation system
- Fraud detection pipeline

Skills developed:

- End-to-end ML engineering
- Data pipeline development
- Model deployment and serving
- Monitoring and maintenance
- Cloud infrastructure (AWS/GCP/Azure)

2. Case Study Development

Deepen domain expertise:

- Industry-specific case studies
- Real company examples
- Failed projects (what went wrong)
- Success stories (what worked)

Topics:

- Healthcare AI implementations
- Financial services ML systems
- Retail recommendation engines
- Manufacturing quality control
- Autonomous vehicle perception

Skills developed:

- Pattern recognition across industries
- Understanding business context
- Learning from others' mistakes
- Best practices and anti-patterns

3. Interview Preparation

Technical interviews:

- ML system design questions
- Case study analysis
- Take-home assignments
- Coding challenges (model implementation)

Behavioral preparation:

- Project discussions
- Trade-off explanations
- Conflict resolution
- Leadership and collaboration

Practice areas:

- "Design a recommendation system for Netflix"
- "How would you detect fraud in real-time?"
- "Build a model to predict house prices"
- "Explain your past ML project"

4. Advanced LLM Workflows

Deep dive into LLMs:

- Advanced prompt engineering
- Fine-tuning techniques
- RAG system optimization
- LLM evaluation and testing
- Cost optimization strategies
- Multi-agent systems
- Tool use and function calling

Topics:

- When to fine-tune vs prompt
- Building production RAG systems
- Evaluating LLM outputs
- Handling hallucinations
- Reducing latency and cost
- Security and safety considerations

Final Thoughts & Summary

The Journey So Far:

Days 1-7: Built foundation

- ML fundamentals
- Model training and evaluation
- Avoiding pitfalls (overfitting, leakage)

Days 8-9: Learned design principles

- Choosing right approach
- Architecture trade-offs
- Cost and latency considerations

Day 10: Connected everything

- AI domains and real-world applications
- Problem → Solution mapping
- End-to-end system thinking

The Core Framework (Memorize This):

1. Understand the problem

- What are we trying to achieve?

- What does success look like?

- What are the constraints?
2. Identify the domain

- Computer Vision (images/video)

- NLP (text)

- Time Series (temporal data)

- Tabular ML (structured data)

- LLM (complex language tasks)
3. Choose the technique

- Match problem to specific method

- Consider alternatives

- Justify choice
4. Design the system

- Data pipeline

- Model architecture

- Deployment approach

- Monitoring strategy
5. Evaluate and iterate

- Define metrics

- Measure performance

- Continuous improvement

Key Principles to Remember:

1. Start Simple

- Rules → Classical ML → Deep Learning → LLM
- Add complexity only when justified
- Simplicity aids debugging and maintenance

2. Data Quality > Model Complexity

- Clean, relevant data beats fancy algorithms
- 80% of work is data preparation
- Garbage in, garbage out

3. Business Context Matters

- Optimize for business metrics, not just accuracy

- Understand costs of different errors
- Consider operational constraints

4. Design for Production

- Cost and latency from day one
- Monitoring and maintenance plans
- Graceful degradation and fallbacks
- Clear escalation paths

5. Continuous Learning

- AI field evolves rapidly
- Stay updated on new techniques
- Learn from failures and successes
- Build portfolio of projects

Resources for Continued Learning:

Datasets:

- Kaggle: kaggle.com/datasets
- UCI ML Repository
- Google Dataset Search
- Hugging Face Datasets

Learning:

- Fast.ai courses (practical deep learning)
- Andrew Ng's ML courses (Coursera)
- Hugging Face NLP course
- Papers with Code (latest research)

Communities:

- Reddit: [r/MachineLearning](https://www.reddit.com/r/MachineLearning), [r/datascience](https://www.reddit.com/r/datascience)
- Twitter: Follow ML researchers and practitioners
- LinkedIn: Connect with AI professionals
- Local meetups and conferences

Practice:

- Kaggle competitions
- Personal projects
- Open source contributions
- Build portfolio on GitHub

Your Next Steps:

Week 1-2:

- Review Day 10 materials
- Complete hands-on assessment
- Pick one project to build
- Set up development environment

Week 3-4:

- Build first end-to-end project
- Deploy to cloud
- Add monitoring
- Document your work

Month 2-3:

- Build 2-3 more projects (different domains)
- Participate in Kaggle competition
- Write blog posts about learnings
- Update resume/portfolio

Month 4-6:

- Start interviewing
- Contribute to open source
- Network with AI professionals
- Consider specializing in one domain

Final Encouragement:

You now have the **framework** and **knowledge** to:

- Analyze any AI problem
- Propose sensible solutions
- Design production systems
- Communicate effectively about AI

This is not the end—it's the beginning of your AI journey.

The field is vast, evolving, and full of opportunities. Stay curious, keep building, and never stop learning.

Good luck! You're ready for real-world AI challenges.

End of Day 10 Teaching Presentation

Quick Reference Guide

AI Domain Selection Cheat Sheet:

Data Type	→ AI Domain
Images/Video	→ Computer Vision
Text	→ NLP or LLM
Time-stamped data	→ Time Series
Tables/CSV	→ Classical ML
Complex documents	→ NLP + LLM (RAG)

Model Selection Cheat Sheet:

Task	→ Model
Image Classification	→ CNN, ResNet, EfficientNet
Object Detection	→ YOLO, Faster R-CNN
OCR	→ Tesseract, Cloud APIs
Text Classification	→ TF-IDF + LogReg, BERT
Named Entity Recognition	→ spaCy, BERT-NER
Search/Retrieval	→ Embeddings + Vector DB
Time Series Forecast	→ ARIMA, Prophet, XGBoost

Anomaly Detection	→ Isolation Forest, Autoencoder
Binary Classification	→ Logistic Regression, XGBoost
Multi-class Classification	→ Random Forest, XGBoost
Regression	→ Linear Regression, XGBoost
Chatbot	→ Intent Classifier or RAG
Q&A System	→ RAG (Retrieval-Augmented Gen)

Cost Guidelines:

Approach	Development	Operation (monthly)
Rules	Low (\$1K)	Negligible
Classical ML	Low (\$2-5K)	Low (\$100-500)
Traditional NLP	Medium (\$5-10K)	Low (\$200-1K)
Deep Learning	High (\$10-50K)	Medium (\$1-5K)
LLM (API)	Low (\$2K)	High (\$1-10K+)
LLM (Fine-tuned)	Very High (\$50K+)	Medium (\$2-5K)

Latency Benchmarks:

Model Type	Typical Latency
Rules	<1 ms
Classical ML	1-50 ms
Traditional NLP	5-100 ms
CNN (image)	50-200 ms
LLM (API)	1000-5000 ms
LLM (self-hosted)	500-3000 ms

Performance Expectations:

Task	Good Accuracy
Image Classification	85-95%
Object Detection	70-90% mAP
OCR (printed)	95-99%
Sentiment Analysis	80-90%
Spam Detection	95-99%
Time Series Forecast	MAE within 5-10% of mean
Fraud Detection	85-95% recall, 70-90% precision
Churn Prediction	75-85% recall

This concludes the comprehensive Day 10 lecture notes. You now have detailed coverage of all AI domains, industry use cases, practical examples, and implementation guidance. Use this as your reference guide for real-world AI projects!

Day 10 Lecture Notes: Core AI Domains, Industry Use Cases & End-to-End Case Studies

Slide 1: Title - Introduction

Day 10: Core AI Domains & Industry Use Cases From Problem Understanding to Real-World AI Systems

Welcome to Day 10!

This is a **capstone-style consolidation day** where everything you've learned comes together. Today is intentionally dense and technical because we're transitioning you from learners to **practitioners** who can design real AI solutions.

What makes Day 10 special:

- This is where theory meets practice
- You'll see how all the concepts connect
- You'll learn to speak the language of AI professionals

- You'll be prepared for real projects and interviews

Slide 2: Why Day 10 Exists

What You've Learned So Far

Let's recap your journey to this point:

Weeks 1-7: Foundations

- **ML fundamentals:** How machine learning works
- **EDA (Exploratory Data Analysis):** Understanding your data before modeling
- **Modeling:** Building predictive models
- **Evaluation:** Measuring model performance with metrics

Week 8-9: Advanced Concepts

- **Validation techniques:** Ensuring your models generalize
- **Overfitting and leakage:** Avoiding critical mistakes
- **Model selection:** Choosing the right approach
- **Cost/architecture trade-offs:** Designing production-ready systems

The Big Question We Answer Today:

"Where does each AI technique actually fit in industry?"

Until now, you've learned **what** these techniques are and **how** they work. Today you learn **when** and **where** to apply them in real business scenarios.

Why This Matters:

In your career, you'll face questions like:

- "We have medical images that need analysis" → Which AI domain?
- "We need to extract information from contracts" → Which techniques?
- "Can you build a recommendation system?" → What approach?

Day 10 gives you the mental framework to answer these confidently.

Slide 3: The Big Picture - AI Domains

The Five Major AI Domains

Most real-world AI systems fall into one or more of these five domains:

1. Computer Vision (CV)

- **What it does:** Enables machines to understand images and videos
- **Common tasks:** Image classification, object detection, facial recognition
- **Example industries:** Healthcare (medical imaging), Manufacturing (quality control), Security (surveillance)

2. Natural Language Processing (NLP)

- **What it does:** Enables machines to understand and process human language
- **Common tasks:** Text classification, sentiment analysis, translation
- **Example industries:** Customer service, Legal, Finance

3. Time Series Forecasting

- **What it does:** Predicts future values based on historical time-ordered data

- **Common tasks:** Demand forecasting, stock prediction, anomaly detection
- **Example industries:** Retail, Finance, Manufacturing

4. Regression & Classification (Tabular ML)

- **What it does:** Makes predictions from structured data in tables
- **Common tasks:** Risk scoring, price prediction, churn prediction
- **Example industries:** Banking, Insurance, E-commerce

5. Large Language Models (LLMs)

- **What it does:** Understands and generates human-like text at scale
- **Common tasks:** Chatbots, document summarization, code generation
- **Example industries:** Tech, Customer support, Content creation

The Critical Insight:

Understanding the domain determines the solution.

When someone presents a business problem, your first job is to identify which domain(s) it falls into. This immediately narrows down your technical choices.

Example mapping:

Business Problem: "Analyze customer reviews for sentiment"

↓

Domain: Natural Language Processing

↓

Technique: Text classification

↓

Model: TF-IDF + Logistic Regression or BERT

PART 1: COMPUTER VISION

Slide 4: What Is Computer Vision?

Definition

Computer Vision (CV) is the field of AI that enables machines to derive meaningful information from visual inputs like images and videos.

What Computer Vision Enables Machines To Do:

1. Understand Images and Videos

- Recognize what objects are present
- Understand spatial relationships
- Interpret scenes and contexts

2. Detect Objects, Patterns, and Text

- Find specific items in images
- Identify recurring visual patterns
- Read and extract text from images (OCR)

3. Make Decisions From Visual Data

- Quality control: Is this product defective?
- Medical diagnosis: Is there a tumor in this X-ray?
- Security: Is this an authorized person?

Data Types in Computer Vision:

Images:

- Static visual data
- 2D arrays of pixels
- Each pixel has color information (RGB values)
- Example: Photographs, X-rays, satellite images

Videos:

- Sequential frames of images
- Add temporal dimension
- Example: Security footage, traffic cameras, sports recordings

How Humans vs Machines See Images:

Human perspective:

- We see a cat sitting on a couch
- We understand context immediately
- Recognition is natural and instantaneous

Machine perspective:

- Sees millions of numbers (pixel values)
- Must learn patterns from examples
- Needs training to recognize "cat" or "couch"

Example: A 224×224 pixel color image = 150,528 numbers ($224 \times 224 \times 3$ color channels)

The Challenge:

Teaching machines to extract meaning from these numbers requires sophisticated algorithms and large amounts of training data.

Slide 5: Image Classification

Task Definition:

Image Classification assigns a single label to an entire image from predefined categories.

Simple analogy: Looking at a photo and saying "This is a cat" or "This is a dog."

Real-World Examples:

Example 1: Manufacturing Quality Control

- **Input:** Photo of product coming off assembly line
- **Output:** "Defective" or "Non-defective"
- **Business value:** Automated quality inspection, reducing human error and labor costs

Example 2: Medical Imaging

- **Input:** Chest X-ray scan
- **Output:** "Pneumonia" or "No pneumonia"
- **Business value:** Faster diagnosis, assisting radiologists, early detection

Example 3: Agriculture

- **Input:** Photo of plant leaf
- **Output:** "Healthy", "Diseased (Type A)", "Diseased (Type B)"

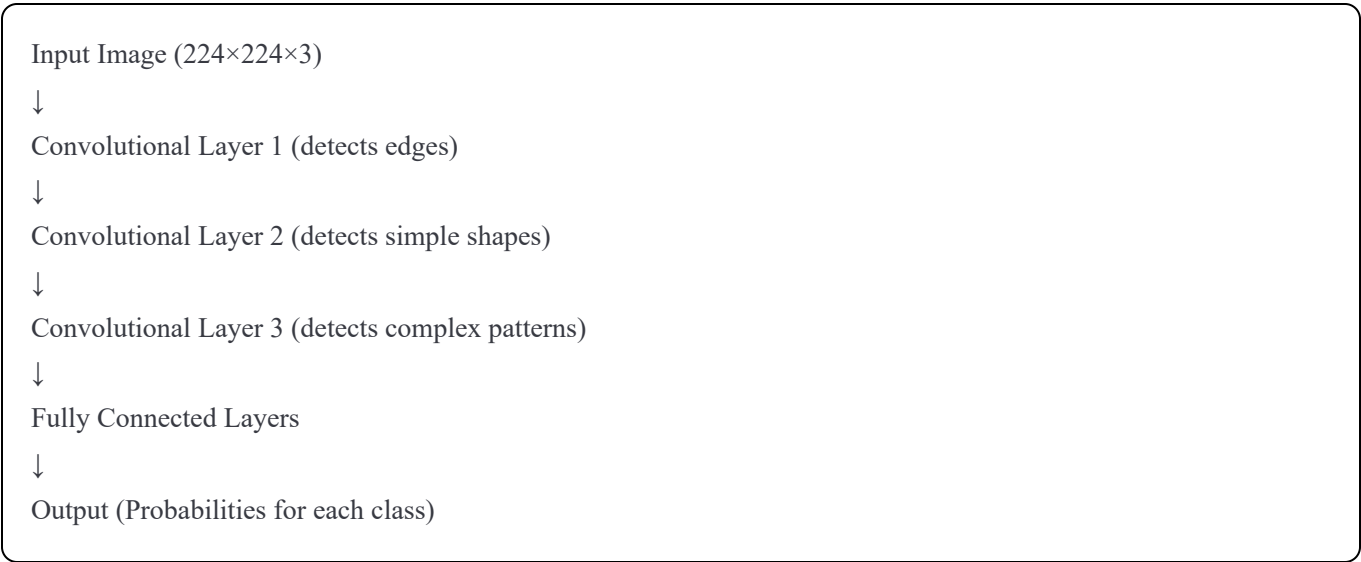
- **Business value:** Early disease detection, reducing crop loss

Common Models for Image Classification:

1. Convolutional Neural Networks (CNNs)

- **What they are:** Deep learning models specifically designed for images
- **How they work:** Learn hierarchical features (edges → shapes → objects)
- **Architecture:** Multiple convolutional layers that detect patterns

Example CNN architecture:



2. Transfer Learning (ResNet, EfficientNet)

- **What it is:** Using pre-trained models instead of training from scratch
- **Why it's powerful:** These models are already trained on millions of images

Transfer Learning Explained:

The Problem: Training a CNN from scratch requires:

- Millions of labeled images
- Weeks of training on expensive GPUs
- Deep expertise in model architecture

The Solution: Transfer Learning

- Take a model already trained on ImageNet (1.2M images, 1000 classes)
- Remove the last layer
- Add your own classification layer
- Train only the final layer on your specific task

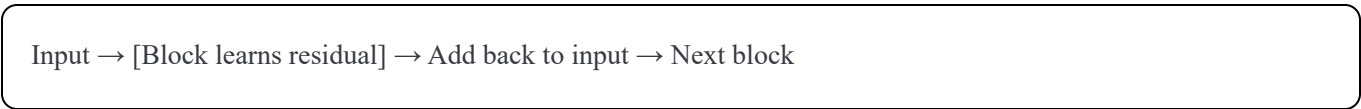
ResNet (Residual Network):

- Introduced in 2015 by Microsoft Research
- Revolutionary architecture with "skip connections"
- Versions: ResNet18, ResNet50, ResNet101, ResNet152
- ResNet50 has 50 layers but trains efficiently due to residual blocks

How Residual Blocks Work: Traditional networks: Each layer must learn the complete transformation



ResNet: Each block learns just the "residual" (the difference)



Why this matters:

- Solves vanishing gradient problem

- Enables training of very deep networks (100+ layers)
- Better accuracy with efficient training

EfficientNet:

- State-of-the-art architecture (2019)
- Balances network depth, width, and resolution
- Achieves better accuracy with fewer parameters
- More efficient than ResNet for similar performance

Dataset Example:

Chest X-Ray Pneumonia Dataset (Kaggle)

- **Link:** <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>
- **Contents:** 5,863 X-Ray images
- **Classes:** Pneumonia, Normal
- **Use case:** Training models to assist radiologists in diagnosis
- **Split:** Training set, validation set, test set

Implementation Example:

```
python

from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model

# Load pre-trained ResNet50 (without top classification layer)
base_model = ResNet50(weights='imagenet', include_top=False,
                        input_shape=(224, 224, 3))

# Freeze base model layers (don't retrain them)
base_model.trainable = False

# Add custom classification layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
predictions = Dense(2, activation='softmax')(x) # 2 classes: pneumonia/normal

# Create final model
model = Model(inputs=base_model.input, outputs=predictions)

# Compile and train
model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])
```

Training approach:

1. Freeze pre-trained layers (use learned features)
2. Train only your new classification layer
3. Optionally: Fine-tune by unfreezing some top layers

Key Takeaways:

- Image classification = one label per image
 - Transfer learning dramatically reduces training requirements
 - ResNet and EfficientNet are industry-standard architectures
 - Can achieve 90-95%+ accuracy with relatively small datasets (1,000-10,000 images)
-

Slide 6: Object Detection

Task Definition:

Object Detection goes beyond classification—it:

1. **Detects** multiple objects in an image
2. **Localizes** where each object is (draws bounding boxes)
3. **Classifies** what each object is

Key difference from classification:

- Classification: "This image contains a car" ✓
- Object Detection: "There are 3 cars at these specific locations" ✓✓✓

Real-World Examples:

Example 1: Traffic Management

- **Input:** Video feed from traffic camera
- **Output:** Detects and counts vehicles, pedestrians, cyclists
- **Bounding boxes:** Draws rectangles around each detected object
- **Business value:**
 - Traffic flow analysis
 - Accident detection
 - Smart traffic light timing
 - Congestion monitoring

Example 2: Retail Analytics

- **Input:** Store surveillance camera footage
- **Output:** Detects and tracks people in store
- **Metrics:**
 - Number of customers in each section
 - Dwell time in areas
 - Traffic patterns
- **Business value:**
 - Optimize store layout
 - Staff allocation
 - Marketing effectiveness
 - Theft prevention

Example 3: Autonomous Vehicles

- **Input:** Real-time camera feed from self-driving car
- **Output:** Detects pedestrians, vehicles, traffic signs, lane markings
- **Critical requirement:** Real-time detection (<100ms latency)
- **Business value:** Safe navigation

Common Models for Object Detection:

1. YOLO (You Only Look Once)

What makes YOLO special:

- **Single-pass detection:** Processes entire image in one forward pass
- **Real-time performance:** Can process 30-60 frames per second

- **End-to-end training:** Predicts bounding boxes and classes simultaneously

How YOLO Works:

Step 1: Grid Division

Divide image into $S \times S$ grid (e.g., $7 \times 7 = 49$ cells)
Each grid cell is "responsible" for detecting objects whose center falls in that cell

Step 2: Bounding Box Prediction

Each cell predicts:

- B bounding boxes (typically $B=2$)
- Each box has 5 values: (x, y, width, height, confidence)
- Confidence = probability object exists $\times$ accuracy of box

Step 3: Class Prediction

Each cell also predicts:

- Probability distribution over C classes
- $P(\text{class_i} \mid \text{object exists})$

Step 4: Non-Maximum Suppression (NMS)

Problem: Multiple boxes might detect same object
Solution: Keep only the box with highest confidence, remove overlapping boxes

Visual Example:

Image of a parking lot with 5 cars

Without NMS:
[Car 1: 15 overlapping boxes]
[Car 2: 12 overlapping boxes]
...

After NMS:
[Car 1: 1 best box (confidence 0.95)]
[Car 2: 1 best box (confidence 0.92)]
...

YOLO Versions Evolution:

- **YOLOv1 (2015):** Original, 45 FPS, revolutionary approach
- **YOLOv2 (2016):** Better accuracy, 67 FPS
- **YOLOv3 (2018):** Multi-scale detection, better small object detection
- **YOLOv4 (2020):** Various optimizations, 62 FPS at higher accuracy
- **YOLOv5 (2020):** PyTorch implementation, user-friendly
- **YOLOv7 (2022):** State-of-the-art speed and accuracy balance
- **YOLOv8 (2023):** Current recommended version, easy to use
- **YOLO11 (2024):** Latest with transformer-based features

Advantages of YOLO:

- ✓ **Speed:** 30-100+ FPS (real-time)
- ✓ **Accuracy:** Competitive with slower methods
- ✓ **Simplicity:** Single network, end-to-end training
- ✓ **Contextual understanding:** Sees whole image at once, fewer false positives

Limitations:

- ✗ Struggles with very small objects

- ✗ Limited to detecting fixed number of objects per grid cell
- ✗ Can have difficulty with clustered objects

2. Faster R-CNN (Region-based Convolutional Neural Network)

How it works:

- **Two-stage approach:**
 1. **Stage 1:** Region Proposal Network suggests potential object locations
 2. **Stage 2:** CNN classifies each proposed region

Comparison:

Feature	YOLO	Faster R-CNN
-----	-----	-----
Speed	Fast (45fps)	Slow (7fps)
Accuracy	Good	Better
Small objects	Struggles	Better
Use case	Real-time	Accuracy-critical

When to use Faster R-CNN:

- Medical imaging (accuracy > speed)
- Detailed analysis scenarios
- When computational resources aren't limited
- Small object detection is critical

When to use YOLO:

- Real-time applications (video surveillance, robotics)
- Autonomous vehicles
- Live sports analysis
- When speed is critical

Dataset Example:

COCO128 Dataset (Kaggle/Ultralytics)

- **Link:** <https://www.kaggle.com/datasets/ultralytics/coco128>
- **Contents:** 128 images from COCO dataset (subset for quick training)
- **Classes:** 80 object categories (person, bicycle, car, etc.)
- **Format:** Images + annotations with bounding boxes
- **Use case:** Training and prototyping object detection models

Full COCO Dataset:

- 330K images
- 1.5 million object instances
- 80 object categories
- Industry-standard benchmark

Implementation Example (YOLOv8):

```
python
```

```
from ultralytics import YOLO

# Load pre-trained YOLOv8 model
model = YOLO('yolov8n.pt') # n = nano (smallest, fastest)

# Train on custom dataset
results = model.train(
    data='custom_dataset.yaml', # Dataset configuration
    epochs=100,
    imgsz=640,
    batch=16
)

# Detect objects in new image
results = model.predict('test_image.jpg', conf=0.5)

# Process results
for result in results:
    boxes = result.boxes # Bounding box coordinates
    for box in boxes:
        print(f'Class: {box.cls}, Confidence: {box.conf}, BBox: {box.xyxy}')
```

Performance Metrics:

Mean Average Precision (mAP):

- **What it measures:** Overall detection accuracy
- **mAP@0.5:** Average precision at 50% IoU threshold
- **mAP@0.5:0.95:** Average across IoU thresholds from 0.5 to 0.95
- **Higher is better:** mAP of 0.75 = 75% average precision

Intersection over Union (IoU):

IoU = (Area of Overlap) / (Area of Union)

Perfect detection: IoU = 1.0

No overlap: IoU = 0.0

Typical threshold: IoU > 0.5 considered correct detection

Key Takeaways:

- Object detection = find + locate + classify multiple objects
- YOLO is the go-to for real-time applications
- Faster R-CNN when accuracy is paramount
- Transfer learning works here too (use pre-trained models)
- Real-world performance: 80-90% mAP achievable with good datasets

Slide 7: OCR & Document Processing

Task Definition:

OCR (Optical Character Recognition) is the technology that converts images of text into machine-readable text.

Simple explanation: Taking a picture of a document and converting it to editable, searchable text.

Real-World Examples:

Example 1: Invoice Processing

- **Input:** Scanned or photographed invoices
- **OCR extracts:**

- Invoice number
- Date
- Vendor name
- Line items and prices
- Total amount
- **Business value:**
 - Automated accounts payable
 - Eliminates manual data entry
 - Reduces errors
 - Faster processing time
- **Typical workflow:**

Scanned Invoice → OCR → Extract fields → Validate → Import to accounting system

Example 2: ID Card Verification

- **Input:** Photo of driver's license, passport, or ID card
- **OCR extracts:**
 - Name
 - Date of birth
 - ID number
 - Address
 - Expiration date
- **Business value:**
 - Automated KYC (Know Your Customer)
 - Faster onboarding
 - Reduced fraud
 - Better user experience
- **Used by:** Banks, fintech companies, rental services, hotels

Example 3: Historical Document Digitization

- **Input:** Old handwritten or printed documents
- **OCR process:** Convert to digital, searchable text
- **Business value:**
 - Preserve historical records
 - Make archives searchable
 - Enable research
- **Used by:** Libraries, museums, archives

Techniques: OCR + NLP Pipelines

The Complete Pipeline:

Stage 1: Image Preprocessing

- Remove noise and artifacts
- Correct skew and rotation
- Enhance contrast
- Binarization (convert to black & white)

Stage 2: Text Detection

- Identify regions containing text

- Separate text from images/graphics
- Determine text layout and structure

Stage 3: Character Recognition (OCR)

- Recognize individual characters
- Convert pixels to text
- Handle different fonts and styles

Stage 4: Post-Processing with NLP

- Spell checking and correction
- Entity extraction (names, dates, amounts)
- Validation against expected formats
- Structuring extracted data

Tools and Technologies:

Tesseract OCR:

- **What it is:** Open-source OCR engine by Google
- **Repository:** <https://github.com/tesseract-ocr/tesseract>
- **Capabilities:**
 - Supports 100+ languages
 - Can handle various fonts
 - Produces plain text or searchable PDFs
 - Can be trained on custom fonts

Usage example:

```
python

import pytesseract
from PIL import Image

# Load image
img = Image.open('invoice.jpg')

# Perform OCR
text = pytesseract.image_to_string(img)
print(text)

# Get detailed data (words, positions, confidence)
data = pytesseract.image_to_data(img, output_type=pytesseract.Output.DICT)
```

Commercial Solutions:

- **Google Cloud Vision API:** High accuracy, cloud-based
- **AWS Textract:** Specialized for documents, extracts tables
- **Azure Form Recognizer:** Pre-trained for invoices, receipts
- **ABBYY FineReader:** Enterprise-grade OCR

Industry Use Cases:

1. Finance

- Check processing
- Invoice automation
- Expense report processing
- Financial statement analysis

- Tax form processing

2. Government

- Passport/ID verification
- Legal document processing
- Census data digitization
- Permit applications
- DMV records

3. Healthcare

- Medical record digitization
- Prescription reading
- Insurance claim forms
- Lab report processing
- Patient intake forms

Challenges in OCR:

1. Image Quality:

- Low resolution images
- Poor lighting
- Blurred or distorted text
- **Solution:** Image preprocessing, enhancement

2. Complex Layouts:

- Multi-column documents
- Tables and forms
- Mixed text and images
- **Solution:** Layout analysis, document segmentation

3. Handwriting:

- Variable handwriting styles
- Cursive writing
- **Solution:** Specialized handwriting recognition models (HWR)

4. Languages and Scripts:

- Multiple languages in one document
- Non-Latin scripts
- **Solution:** Multi-language models, language detection

Modern Advances:

Deep Learning for OCR:

- CNNs for character recognition
- RNNs for sequence modeling
- Attention mechanisms for context
- **Result:** 95-99% accuracy on printed text

Document AI:

- Understands document structure
- Extracts key-value pairs

- Handles semi-structured data
- Example: "Invoice No: 12345" → {"invoice_number": "12345"}

Key Takeaways:

- OCR converts images of text to machine-readable format
 - Essential for automating document-heavy workflows
 - Combine OCR with NLP for intelligent extraction
 - Tesseract is excellent open-source option
 - Cloud APIs provide state-of-the-art accuracy
 - 90-99% accuracy achievable on good quality documents
-

Slide 8: CV Industry Use Cases

Industries Using Computer Vision Heavily:

1. Manufacturing (Quality Inspection)

Application: Automated Visual Inspection

Traditional Process:

- Human inspectors examine products
- Look for defects, scratches, misalignment
- Slow, tiring, prone to inconsistency

CV Solution:

- Cameras capture images of every product
- CNN models detect defects in milliseconds
- Categorize defect types automatically
- Alert system for defective items

Example: Smartphone Screen Inspection



Benefits:

- 100% inspection (vs 5-10% sampling)
- Consistent quality standards
- 24/7 operation
- Detailed defect analytics
- ROI: Payback in 6-12 months

Real Numbers:

- Human inspector: 500-1000 products/hour
- CV system: 50,000+ products/hour
- Accuracy: 99%+ vs 85-90% human accuracy

2. Healthcare (Medical Imaging)

Applications:

A. Disease Diagnosis

- **X-rays:** Detect pneumonia, tuberculosis, fractures
- **MRI/CT scans:** Identify tumors, lesions
- **Retinal images:** Diabetic retinopathy screening
- **Skin images:** Melanoma detection

Example: Chest X-ray Analysis

Patient X-ray → CNN model →

Outputs:

- "Pneumonia detected (confidence: 92%)"
- "Location: Right lower lobe"
- "Severity: Moderate"

B. Medical Image Segmentation

- Precisely outline organs, tumors, blood vessels
- Assist in surgical planning
- Monitor disease progression

Benefits:

- Faster diagnosis (minutes vs hours/days)
- Second opinion for radiologists
- Early detection saves lives
- Reduce radiologist workload
- 24/7 availability

Real-World Impact:

- **Stanford Study:** CNN matched expert radiologists in pneumonia detection
- **Google DeepMind:** AI detects 50+ eye diseases from retinal scans
- **PathAI:** 99% accuracy in cancer diagnosis from pathology slides

Important Note: CV assists doctors; it doesn't replace them. Final diagnosis remains with medical professionals.

3. Security & Surveillance

Applications:

A. Facial Recognition

- Access control to buildings
- Airport security
- Criminal identification
- Missing person search

B. Anomaly Detection

- Unusual behavior detection
- Unattended baggage
- Crowd monitoring
- Perimeter breach

C. License Plate Recognition (ANPR)

- Toll collection
- Parking management
- Traffic law enforcement
- Vehicle tracking

Example: Smart Surveillance System

Multiple cameras → Real-time CV analysis →

Detections:

- Person loitering >10 minutes
- Unattended bag detected
- Restricted area breach
- Face matches watchlist

→ Alert security personnel

Benefits:

- 24/7 monitoring without fatigue
- Instant alerts for threats
- Searchable video archives
- Reduced security personnel needs
- Faster response times

Privacy Considerations:

- Must comply with regulations (GDPR, local laws)
- Secure storage of biometric data
- Transparent usage policies
- Opt-in/opt-out mechanisms where appropriate

4. Retail (Shelf Monitoring)

Applications:

A. Inventory Management

- Detect out-of-stock items
- Monitor product placement
- Track competitor products
- Verify planogram compliance

B. Customer Analytics

- Heat maps of customer movement
- Dwell time analysis
- Demographics (age, gender estimation)
- Emotion detection (engagement level)

C. Checkout Automation

- Amazon Go-style stores
- Automated product recognition
- No cashiers needed

Example: Smart Shelf Monitoring

Camera above shelf → CV model analyzes →

Detects:

- Product A: Out of stock (red)
 - Product B: Low stock (yellow)
 - Product C: Misplaced (wrong shelf)
 - Product D: Stock level OK (green)
- Alert store staff or auto-reorder

Benefits:

- Real-time inventory visibility
- Reduce out-of-stock situations (+5-10% sales)
- Optimize store layout
- Understand customer behavior
- Labor cost savings

ROI Example:

- **Problem:** 8% of desired items out of stock → lost sales
- **Solution:** CV monitoring → reduce to 2% out-of-stock
- **Result:** 6% sales increase = significant revenue

Why CV Systems Are High-Impact:

1. Data-Intensive (Challenge)

- Require thousands to millions of labeled images
- Data annotation is expensive and time-consuming
- Domain expertise needed for labeling (e.g., medical images)

2. High-Impact (Opportunity)

- Automate what was impossible before
- Scale beyond human capacity
- Consistent 24/7 performance
- ROI often measured in months, not years

3. Transfer Learning Helps:

- Pre-trained models reduce data requirements
- Can achieve good results with 1,000-10,000 images
- Fine-tuning is faster and cheaper

4. Getting Started:

- Start with proof-of-concept (small dataset)
- Validate accuracy and business value
- Scale gradually
- Partner with CV experts initially

Key Takeaways:

- CV is transforming multiple industries
- Applications range from quality control to medical diagnosis
- High impact but requires significant data and expertise
- Transfer learning makes CV more accessible
- ROI can be substantial when applied correctly
- Must consider ethical and privacy implications

PART 2: NATURAL LANGUAGE PROCESSING (NLP)

Slide 9: What Is NLP?

Definition:

Natural Language Processing (NLP) is the branch of AI that helps computers understand, interpret, and generate human language.

What NLP Focuses On:

1. Understanding Text

- Read and comprehend written language
- Extract meaning from sentences
- Understand context and nuance

2. Extracting Meaning

- Identify key information
- Determine sentiment (positive/negative/neutral)
- Recognize entities (names, places, dates)

3. Structuring Unstructured Language

- Convert free-form text into organized data
- Extract structured information from documents
- Create databases from text sources

Data Type:

Text - The raw material of NLP

- Social media posts
- Customer reviews
- Emails and documents
- News articles
- Chat conversations
- Product descriptions
- Legal contracts
- Medical records

Why NLP Is Challenging:

1. Ambiguity

"I saw her duck"

- Meaning 1: I saw her pet duck
- Meaning 2: I saw her quickly lower her head

2. Context Dependency

"That's sick!"

- In medical context: Something is wrong
- In slang: That's really cool!

3. Sarcasm and Irony

"Great, another meeting. Just what I needed!" (Actually means the opposite)

4. Multiple Languages

- Grammar rules differ
- Word order matters differently
- Idioms don't translate literally

5. Informal Language

- Misspellings: "u r rite"
- Abbreviations: "LOL", "ASAP"
- Emojis: 😊, 😞, 🔥

Evolution of NLP:

Traditional NLP (Pre-2015):

- Rule-based systems
- Statistical methods
- Bag of words, TF-IDF
- Simpler but limited

Modern NLP (2015-Present):

- Deep learning models
- Word embeddings (Word2Vec, GloVe)
- Transformers (BERT, GPT)
- Much more powerful but complex

Current State:

- LLMs can understand context deeply
- Multi-lingual models
- Can generate human-like text
- Still struggles with nuance sometimes

The NLP Pipeline:

Raw Text → Preprocessing → Feature Extraction → Model → Output

Example:
"The movie was awesome!"
→ Tokenize, clean
→ Convert to numbers
→ Sentiment classifier
→ "Positive (95% confidence)"

Key Takeaways:

- NLP bridges human language and machine understanding
 - Works with unstructured text data
 - Challenging due to ambiguity and context
 - Essential for any text-heavy application
 - Powers chatbots, search, translation, and more
-

Slide 10: Text Classification

Task Definition:

Text Classification assigns predefined categories or labels to text documents.

Simple analogy: Sorting emails into folders (Spam, Work, Personal).

Real-World Examples:

Example 1: Sentiment Analysis

Task: Determine if text expresses positive, negative, or neutral sentiment

Use Cases:

- **Product reviews:** "This phone is amazing!" → Positive
- **Social media monitoring:** Track brand perception
- **Customer feedback:** Identify unhappy customers quickly
- **Movie reviews:** Aggregate sentiment scores

Example:

Review: "The battery life is terrible and customer service was rude."
→ Model Output: Negative (confidence: 0.92)

Review: "Amazing product! Exceeded my expectations."
→ Model Output: Positive (confidence: 0.96)

Business Value:

- Real-time brand monitoring
- Identify product issues quickly
- Prioritize customer support tickets
- Measure marketing campaign success

Example 2: Spam Detection

Task: Classify emails as spam or not spam

Features that help detection:

- Keywords: "free", "winner", "click here", "urgent"
- Sender reputation
- Email formatting
- Links to suspicious domains
- Grammar and spelling errors

Example:

Email: "Congratulations! You've won \$1,000,000! Click here now!!!"
Features: Urgency, money, suspicious link, multiple exclamation marks
→ Model Output: Spam (confidence: 0.99)

Email: "Meeting tomorrow at 3pm in Conference Room B"
Features: Professional tone, no suspicious keywords
→ Model Output: Not Spam (confidence: 0.98)

Business Value:

- Protect users from scams
- Reduce inbox clutter
- Prevent phishing attacks
- Improve email experience

Other Text Classification Examples:

- **Topic classification:** Categorize news articles (Sports, Politics, Tech, etc.)
- **Language detection:** Identify which language text is written in
- **Intent classification:** Understand what user wants ("book a flight", "check balance")
- **Document categorization:** Sort legal documents, medical records

Techniques:

1. TF-IDF + Classical ML

TF-IDF (Term Frequency-Inverse Document Frequency):

What it measures:

- **Term Frequency (TF):** How often does a word appear in this document?
- **Inverse Document Frequency (IDF):** How rare is this word across all documents?
- **Intuition:** Important words appear frequently in one document but rarely in others

Example:

Document: "The cat sat on the mat. The cat was happy."

TF for "cat": 2/9 (appears 2 times out of 9 words)
TF for "the": 3/9 (very common, less important)

If "cat" appears in 10 out of 1000 documents:
IDF for "cat" = $\log(1000/10)$ = high (rare, important)

If "the" appears in 900 out of 1000 documents:
IDF for "the" = $\log(1000/900)$ = low (common, less important)

TF-IDF for "cat" = $TF \times IDF$ = HIGH (important word!)

Why it's powerful:

- Automatically identifies important words
- Filters out common words ("the", "is", "and")
- No need for manual feature engineering

Complete Pipeline:

Text → TF-IDF Vectorization → Feature Matrix → ML Model → Prediction

Example:
"This movie is great!"
→ [0.0, 0.8, 0.0, 0.9, 0.0, ...] (TF-IDF vector)
→ Logistic Regression
→ Positive Sentiment

Classical ML Models Used:

- **Logistic Regression:** Simple, fast, interpretable
- **Naive Bayes:** Works well for text, assumes word independence
- **Support Vector Machines (SVM):** Good for high-dimensional text data
- **Random Forest:** Ensemble method, robust

2. Embeddings + Classifiers

Word Embeddings:

- Convert words to dense vectors (lists of numbers)
- Similar words have similar vectors

- Capture semantic meaning

Example:

"king" → [0.2, 0.8, -0.3, 0.5, ...]
"queen" → [0.3, 0.7, -0.2, 0.6, ...] (Similar to king)
"car" → [-0.5, 0.1, 0.9, -0.2, ...] (Very different)

Vector similarity: king ≈ queen (related concepts)

Popular Embedding Methods:

- **Word2Vec:** Learn embeddings from context
- **GloVe:** Global vectors for word representation
- **FastText:** Handles out-of-vocabulary words
- **BERT embeddings:** Contextual (word meaning changes by context)

Pipeline with Embeddings:

Text → Tokenize → Word Embeddings → Average/Pool → Neural Network → Prediction

Example:
"Amazing product"
→ ["Amazing", "product"]
→ [[0.8, -0.2, ...], [0.7, 0.3, ...]]
→ Average vector
→ Neural network classifier
→ Positive

Advantages over TF-IDF:

- Captures semantic similarity
- Handles synonyms better
- More robust to word variations
- Better for complex classifications

Dataset Example:

Sentiment140 Dataset (Kaggle)

- **Link:** <https://www.kaggle.com/datasets/kazanova/sentiment140>
- **Contents:** 1.6 million tweets
- **Labels:** Positive (0) and Negative (4) sentiment
- **Features:** Tweet text, username, date
- **Use case:** Training sentiment analysis models

Why this dataset is good:

- Large size (1.6M samples)
- Real-world social media text
- Informal language, slang, emojis
- Challenging (sarcasm, abbreviations)

Implementation Example:

python

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Sample data
texts = [
    "This product is amazing!",
    "Terrible experience, very disappointed",
    "Great quality, highly recommend",
    "Waste of money, do not buy"
]
labels = [1, 0, 1, 0] # 1 = positive, 0 = negative

# TF-IDF vectorization
vectorizer = TfidfVectorizer(max_features=1000)
X = vectorizer.fit_transform(texts)

# Train classifier
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2)
classifier = LogisticRegression()
classifier.fit(X_train, y_train)

# Predict
new_text = ["This is the best purchase ever!"]
new_vec = vectorizer.transform(new_text)
prediction = classifier.predict(new_vec)
print(f'Sentiment: {'Positive' if prediction[0] == 1 else 'Negative'})
```

Performance Metrics:

For Binary Classification (Spam/Not Spam):

- **Accuracy:** Overall correctness
- **Precision:** Of items marked as spam, how many are actually spam?
- **Recall:** Of all actual spam, how many did we catch?
- **F1-Score:** Balance between precision and recall

For Sentiment (Multi-class):

- **Accuracy:** Overall correctness across all classes
- **Confusion Matrix:** See which classes are confused
- **Macro/Micro F1:** Averaged performance across classes

Typical Performance:

- Spam detection: 95-99% accuracy
- Sentiment analysis: 80-90% accuracy (harder due to sarcasm, context)
- Topic classification: 85-95% accuracy

Key Takeaways:

- Text classification = assign predefined labels to text
 - TF-IDF + Logistic Regression is excellent baseline
 - Embeddings capture semantic meaning
 - Can achieve 80-95% accuracy on most tasks
 - Fast to train and deploy (minutes to hours)
 - Wide range of business applications
-

Slide 11: Information Extraction

Task Definition:

Information Extraction (IE) is the process of automatically extracting structured information from unstructured text.

Simple explanation: Finding specific pieces of information in text and organizing them into a structured format.

Real-World Examples:

Example 1: Invoice Field Extraction

Input: Scanned invoice image + OCR text

ACME Corporation
Invoice #: INV-2024-001
Date: January 15, 2024
Bill To: XYZ Company
123 Main Street

Item	Qty	Price	Total
Widget A	10	\$50	\$500
Widget B	5	\$30	\$150

Subtotal: \$650
Tax: \$65
Total: \$715

Information Extraction Output:

```
json

{
  "vendor": "ACME Corporation",
  "invoice_number": "INV-2024-001",
  "date": "2024-01-15",
  "customer": "XYZ Company",
  "items": [
    { "name": "Widget A", "quantity": 10, "unit_price": 50, "total": 500 },
    { "name": "Widget B", "quantity": 5, "unit_price": 30, "total": 150 }
  ],
  "subtotal": 650,
  "tax": 65,
  "total": 715
}
```

Business Value:

- Automated accounts payable
- Eliminate manual data entry (saves hours per day)
- Reduce errors (typos, transposition errors)
- Faster processing and payment
- Better spend analytics

ROI Example:

- Manual processing: 5 minutes per invoice
- 1000 invoices/month = 83 hours/month
- Automated: 30 seconds per invoice with validation
- Savings: ~75 hours/month per processor

Example 2: Resume Parsing

Input: Candidate resume (PDF or Word document)

John Doe
john.doe@email.com | (555) 123-4567 | New York, NY

EXPERIENCE

Senior Data Scientist, Tech Corp (2020-Present)

- Led team of 5 data scientists
- Built ML models for customer churn prediction
- Tech stack: Python, TensorFlow, AWS

EDUCATION

M.S. Computer Science, MIT (2018-2020)
B.S. Mathematics, Harvard (2014-2018)

SKILLS

Python, Machine Learning, Deep Learning, SQL, AWS

Information Extraction Output:

```
json

{
  "name": "John Doe",
  "email": "john.doe@email.com",
  "phone": "(555) 123-4567",
  "location": "New York, NY",
  "experience": [
    {
      "title": "Senior Data Scientist",
      "company": "Tech Corp",
      "duration": "2020-Present",
      "years": 4,
      "responsibilities": [
        "Led team of 5 data scientists",
        "Built ML models for customer churn prediction"
      ],
      "skills": ["Python", "TensorFlow", "AWS"]
    }
  ],
  "education": [
    {
      "degree": "M.S. Computer Science",
      "institution": "MIT",
      "years": "2018-2020"
    },
    {
      "degree": "B.S. Mathematics",
      "institution": "Harvard",
      "years": "2014-2018"
    }
  ],
  "skills": ["Python", "Machine Learning", "Deep Learning", "SQL", "AWS"]
}
```

Business Value:

- Automated candidate screening
- Quick skill matching to job requirements
- Build searchable candidate databases
- Reduce recruiter workload
- Faster time-to-hire

Techniques:

1. Named Entity Recognition (NER)

What is NER? Named Entity Recognition identifies and classifies named entities in text into predefined categories.

Common Entity Types:

- **PER (Person):** Names of people
- **ORG (Organization):** Companies, institutions
- **LOC (Location):** Cities, countries, addresses
- **DATE:** Dates and times
- **MONEY:** Monetary values
- **PERCENT:** Percentages
- **PRODUCT:** Product names
- **EMAIL:** Email addresses
- **PHONE:** Phone numbers

Example:

Text: "Apple Inc. CEO Tim Cook announced new iPhone in Cupertino on September 12, 2023."

NER Output:

- "Apple Inc." → ORGANIZATION
- "Tim Cook" → PERSON
- "iPhone" → PRODUCT
- "Cupertino" → LOCATION
- "September 12, 2023" → DATE

How NER Works:

Traditional Approach:

1. **Rule-based:** Pattern matching, dictionaries
2. **Statistical:** Hidden Markov Models, CRFs
3. **Features:** Capitalization, POS tags, word shape

Modern Approach (Deep Learning):

1. **Word embeddings:** Convert text to vectors
2. **Bidirectional LSTM:** Process text forward and backward
3. **CRF layer:** Ensure valid label sequences
4. **BERT-based:** State-of-the-art, contextual understanding

Popular NER Tools:

- **spaCy:** Fast, production-ready, supports 60+ languages
- **Stanford NER:** Java-based, highly accurate
- **NLTK:** Educational, good for learning
- **Hugging Face Transformers:** State-of-the-art BERT models

spaCy Example:

```
python
```

```
import spacy

# Load pre-trained model
nlp = spacy.load("en_core_web_sm")

# Process text
text = "Microsoft was founded by Bill Gates in Seattle in 1975."
doc = nlp(text)

# Extract entities
for ent in doc.ents:
    print(f"{ent.text:20} → {ent.label_}")

Output:
Microsoft      → ORG
Bill Gates     → PERSON
Seattle        → GPE (Geopolitical Entity)
1975           → DATE
```

2. Rule + ML Pipelines

Why Combine Rules and ML?

- Rules handle well-defined patterns
- ML handles variations and context
- Hybrid approach is more robust

Example: Email Extraction Pipeline

Step 1: Rule-based detection

- Pattern: word@domain.com

- Regex: \b[A-Za-z0-9._%+-]+\@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b

Step 2: ML validation

- Is this a valid email format?

- Is the domain legitimate?

- Context: Does surrounding text make sense?

Step 3: Post-processing

- Remove duplicates

- Validate domain exists

- Check against blacklist

Example: Address Extraction

Step 1: NER detects potential locations

Step 2: Rules identify address patterns

- Pattern: Number + Street Name + City + State + ZIP

- "123 Main St, New York, NY 10001"

Step 3: ML validates

- Is this a complete address?

- Are components in correct order?

- Confidence scoring

Step 4: Standardization

- Convert to standard format

- Geocode if needed

Benefits of Hybrid Approach:

- Higher accuracy than rules or ML alone
- More explainable than pure ML
- Easier to customize and maintain

- Handles edge cases better

Dataset Example:

NER Dataset (Kaggle)

- **Link:** <https://www.kaggle.com/datasets/dataturks/ner-dataset>
- **Contents:** Annotated sentences with entity labels
- **Format:**

Word	Tag
Apple	B-ORG
Inc.	I-ORG
CEO	O
Tim	B-PER
Cook	I-PER

- **Tags:** B- (beginning), I- (inside), O (outside)
- **Use case:** Training custom NER models

Implementation Example:

Training Custom NER with spaCy:

```
python

import spacy
from spacy.training import Example

# Create blank model
nlp = spacy.blank("en")

# Add NER component
ner = nlp.add_pipe("ner")

# Add labels
ner.add_label("PRODUCT")
ner.add_label("PRICE")

# Training data
TRAIN_DATA = [
    ("I bought an iPhone for $999", {
        "entities": [(14, 20, "PRODUCT"), (25, 29, "PRICE")]
    }),
    ("The Samsung Galaxy costs $800", {
        "entities": [(4, 18, "PRODUCT"), (25, 29, "PRICE")]
    })
]

# Train
optimizer = nlp.begin_training()
for epoch in range(10):
    for text, annotations in TRAIN_DATA:
        example = Example.from_dict(nlp.make_doc(text), annotations)
        nlp.update([example], sgd=optimizer)

# Test
doc = nlp("I want to buy a MacBook for $1200")
for ent in doc.ents:
    print(f'{ent.text} → {ent.label_}')
```

Performance Considerations:

Accuracy:

- **Simple entities (names, dates):** 90-95% F1-score
- **Complex entities (addresses, products):** 80-90% F1-score

- **Domain-specific (medical, legal):** Requires custom training

Speed:

- spaCy: 100,000 words/second on CPU
- BERT models: Slower but more accurate
- Rule-based: Fastest but less flexible

Key Takeaways:

- Information extraction converts unstructured text to structured data
 - NER is the core technique for identifying entities
 - Hybrid (rule + ML) approaches work best
 - Applications: Document processing, resume parsing, data extraction
 - spaCy is industry-standard for production use
 - Can achieve 85-95% accuracy with good training data
-

Slide 12: Search & Retrieval

Task Definition:

Search & Retrieval is the task of finding relevant documents or information from a large collection based on a query.

Simple explanation: Like Google search, but for your company's internal documents.

Real-World Examples:

Example 1: Enterprise Search

Scenario: Large company with thousands of internal documents

User Query: "What is our travel expense policy?"

Traditional Search (Keyword-based):

Search for exact words: "travel" AND "expense" AND "policy"

Problems:

- Misses documents with synonyms ("reimbursement", "trip")
- Returns too many irrelevant results
- Can't understand intent

Modern Search (Semantic/Vector-based):

Understands query intent

Finds documents about:

- Travel reimbursement procedures
- Business trip guidelines
- Expense claim forms

Even if exact words don't match!

Results:

1. "Employee Travel and Expense Reimbursement Policy" (99% relevance)
2. "Business Trip Guidelines" (95% relevance)
3. "Expense Report Submission Process" (92% relevance)

Business Value:

- Employees find information faster
- Reduce repetitive questions to HR/Finance
- Improve compliance (people follow correct policies)

- Better knowledge sharing

Example 2: Knowledge Base Q&A

Scenario: Customer support with extensive product documentation

Customer Query: "How do I reset my password?"

System Process:

1. Understand query intent
2. Search knowledge base
3. Find most relevant articles:
 - "Password Reset Guide"
 - "Account Recovery Steps"
 - "Login Troubleshooting"
4. Return top matches with snippets

Benefits:

- 24/7 self-service support
- Instant answers
- Reduce support ticket volume
- Consistent information
- Multilingual support

Impact:

- 30-40% reduction in support tickets
- 70% of queries resolved without human agent
- Faster resolution time
- Better customer satisfaction

Techniques:

1. Vector Embeddings

What are Vector Embeddings?

- Convert text (documents, queries) into numerical vectors
- Similar content has similar vectors
- Enable semantic search (search by meaning, not just keywords)

How It Works:

Document: "Our company offers health insurance and dental coverage"

→ Embedding Model

→ Vector: [0.2, -0.5, 0.8, 0.1, -0.3, ...] (hundreds of dimensions)

Query: "What medical benefits do you provide?"

→ Embedding Model

→ Vector: [0.3, -0.4, 0.7, 0.2, -0.2, ...] (similar to document!)

Similarity = Cosine similarity between vectors

→ High similarity = relevant document!

Popular Embedding Models:

- **Sentence-BERT:** Optimized for semantic similarity
- **OpenAI Ada:** High quality, API-based
- **all-MiniLM-L6-v2:** Fast, open-source, good quality
- **BGE embeddings:** State-of-the-art, multilingual

Why Embeddings Are Powerful:

Query: "canine companions"

Traditional search: Looks for exact words "canine" and "companions"

Embedding search: Understands this is about dogs and finds:

- "dog ownership guide"
- "pet care for puppies"
- "man's best friend tips"

2. Similarity Search

Vector Similarity Metrics:

a) Cosine Similarity (Most common)

Measures angle between vectors

Range: -1 to 1 (1 = identical, 0 = orthogonal, -1 = opposite)

Example:

Query vector: [1, 0, 1]

Doc1 vector: [1, 0, 1] → Cosine similarity = 1.0 (perfect match)

Doc2 vector: [0, 1, 0] → Cosine similarity = 0.0 (unrelated)

b) Euclidean Distance

Measures straight-line distance between vectors

Smaller distance = more similar

c) Dot Product

Measures alignment between vectors

Higher dot product = more similar

Vector Database Architecture:

1. Indexing Phase:

Documents → Embeddings → Store in Vector DB

2. Query Phase:

User Query → Embedding → Search similar vectors → Return top-K documents

3. Optimization:

Use approximate nearest neighbor (ANN) algorithms for speed

- HNSW (Hierarchical Navigable Small World)
- IVF (Inverted File Index)
- LSH (Locality-Sensitive Hashing)

Popular Vector Databases:

- **Pinecone:** Managed, cloud-native, easy to use
- **Weaviate:** Open-source, feature-rich
- **Chroma:** Lightweight, embedded database
- **Qdrant:** Fast, Rust-based
- **FAISS (Facebook):** Library for efficient similarity search
- **Milvus:** Scalable, open-source

Implementation Example:

Basic Semantic Search with Sentence-BERT:

python

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Load pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Documents
documents = [
    "Python is a programming language",
    "Machine learning is a subset of AI",
    "Dogs are loyal pets",
    "Neural networks are used in deep learning"
]

# Create embeddings
doc_embeddings = model.encode(documents)

# User query
query = "Tell me about artificial intelligence"
query_embedding = model.encode([query])

# Calculate similarities
similarities = cosine_similarity(query_embedding, doc_embeddings)[0]

# Rank documents
ranked_indices = np.argsort(similarities)[::-1]

# Display results
for idx in ranked_indices:
    print(f"Score: {similarities[idx]:.3f} | {documents[idx]}")
```

Output:

Score: 0.657 | Machine learning is a subset of AI

Score: 0.421 | Neural networks are used in deep learning

Score: 0.223 | Python is a programming language

Score: 0.089 | Dogs are loyal pets

Production System with Pinecone:

python

```
import pinecone
from sentence_transformers import SentenceTransformer

# Initialize
pinecone.init(api_key="your-api-key")
index = pinecone.Index("document-search")

# Load embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Index documents
documents = {
    "doc1": "Travel expense policy document...",
    "doc2": "HR benefits overview...",
    # ... more documents
}

for doc_id, text in documents.items():
    embedding = model.encode(text).tolist()
    index.upsert([(doc_id, embedding, {"text": text})])

# Search
query = "What is the expense policy?"
query_embedding = model.encode(query).tolist()
results = index.query(query_embedding, top_k=5, include_metadata=True)

# Display results
for match in results['matches']:
    print(f"Score: {match['score']:.3f}")
    print(f"Text: {match['metadata']['text'][:100]} ...")
```

Reference:

Pinecone Learning Resources

- **Link:** <https://www.pinecone.io/learn/vector-search/>
- **Topics covered:**
 - What is vector search
 - How embeddings work
 - Building production systems
 - Performance optimization
 - Use case examples

Advanced Topics:

Hybrid Search: Combine keyword search with semantic search for best results

Query: "iPhone 15 Pro review"

Keyword component: Find exact matches for "iPhone 15 Pro"

Semantic component: Understand intent (looking for reviews, opinions)

Combined ranking:

- 50% keyword score
- 50% semantic score

→ Better results than either alone

Metadata Filtering:

Query: "travel policy"

Filters:

- Department: "Finance"
- Date: After 2023
- Document type: "Policy"

Process:

1. Apply filters (narrow down documents)
2. Semantic search within filtered set
3. Return most relevant

Re-ranking:

Step 1: Fast semantic search (retrieve top 100 documents)

Step 2: Re-rank with more sophisticated model

Step 3: Return top 10 to user

Why: Balance speed and accuracy

Performance Metrics:

Relevance Metrics:

- **Recall@K:** Of all relevant documents, how many are in top-K results?
- **Precision@K:** Of top-K results, how many are relevant?
- **MRR (Mean Reciprocal Rank):** Average of 1/rank of first relevant result
- **NDCG (Normalized Discounted Cumulative Gain):** Rewards relevant results at top

Speed Metrics:

- **Query latency:** Time to return results (target: <100ms)
- **Indexing speed:** Documents indexed per second
- **Throughput:** Queries per second

Typical Performance:

- **Relevance:** 70-85% Recall@10 (top 10 results contain 70-85% of relevant docs)
- **Speed:** 10-50ms query latency for millions of documents
- **Scale:** Handle 1M-1B documents efficiently

Key Takeaways:

- Modern search uses vector embeddings for semantic understanding
- Vector databases enable fast similarity search at scale
- Hybrid approaches (keyword + semantic) work best
- Essential for enterprise search, Q&A systems, recommendation
- Pinecone, Weaviate, Chroma are production-ready solutions
- Can achieve 70-85% relevance with good embeddings

Slide 13: Conversational AI & Chatbots

Overview:

Conversational AI enables machines to have natural, human-like conversations through text or voice.

Real-World Examples:

Example 1: Customer Support Bots

- **Scenario:** E-commerce website with 24/7 support needs

- **Capabilities:**
 - Answer FAQ questions
 - Track orders
 - Process returns
 - Escalate to human when needed

Example 2: Internal Knowledge Assistants

- **Scenario:** Large company with complex policies and procedures
- **Capabilities:**
 - Answer employee questions about HR policies
 - Help with IT troubleshooting
 - Provide training resources
 - Schedule meetings

Techniques:

1. Intent Classification (Classical ML)

What is Intent? The user's goal or purpose behind their message.

Example Intents:

- `check_order_status`
- `request_refund`
- `ask_hours`
- `complaint`

How It Works:

User: "When will my package arrive?"
→ Intent Classifier
→ Intent: `check_order_status` (confidence: 0.95)
→ Extract entities: None needed
→ Response template: "Let me check your order status. What's your order number?"

Training Data:

Intent: `check_order_status`
Examples:
- "Where is my order?"
- "When will I receive my package?"
- "Track my delivery"
- "Shipping status"

Intent: `request_refund`
Examples:
- "I want my money back"
- "How do I get a refund?"
- "Cancel my order and refund"
- "Return this product"

Classical ML Approach:

- Use TF-IDF or embeddings as features
- Train multi-class classifier
- Fast and efficient
- Works well for 10-50 intents

2. LLM-Based Chatbots

Modern Approach: Use Large Language Models (GPT, Claude) for more flexible conversations

Advantages:

- Natural, human-like responses
- Handle unexpected queries
- No need to predefine all intents
- Context-aware conversations
- Can explain reasoning

Challenges:

- More expensive (API costs)
- Slower response time
- Can hallucinate (make up information)
- Requires careful prompt engineering

3. RAG Systems (Retrieval-Augmented Generation)

Best of Both Worlds: Combine retrieval (search) with generation (LLM)

How RAG Works:

1. User asks question
2. Search company knowledge base for relevant documents
3. Send retrieved documents + question to LLM
4. LLM generates answer based on actual company information
5. Return answer with source citations

Example:

User: "What's the return policy for electronics?"

Step 1: Search knowledge base
→ Finds: "Electronics Return Policy" document

Step 2: Send to LLM with prompt:
"Based on this document: [policy text],
answer: What's the return policy for electronics?"

Step 3: LLM generates accurate answer:
"For electronics, you have 30 days to return unused items
with original packaging. Restocking fee of 15% applies.
Opened items: 14-day return window..."

Step 4: Include citation:
"Source: Electronics Return Policy (Updated Jan 2024)"

Why RAG is Better:

- ✓ Grounded in actual company information (no hallucination)
- ✓ Always up-to-date (searches current documents)
- ✓ Can cite sources (build trust)
- ✓ More accurate than pure LLM
- ✓ More flexible than pure rules

Key Takeaways:

- Multiple approaches available (classical ML, LLMs, RAG)
- RAG is becoming industry standard
- Balance cost, accuracy, and flexibility
- Always have human escalation path
- Monitor and improve continuously

PART 3: TIME SERIES FORECASTING

Slide 14: What Is Time Series Data?

Definition:

Time Series Data is a sequence of data points indexed in time order.

Key Characteristic: The order of data points matters!

Properties:

Time-Indexed:

- Each data point has a timestamp
- Regular intervals (hourly, daily, monthly)
- Or irregular intervals (event-based)

Order Matters:

- You can't shuffle time series data
- Past influences future
- Temporal dependencies exist

Examples:

Example 1: Sales Over Time

Date	Sales
2024-01-01	\$5,000
2024-01-02	\$5,500
2024-01-03	\$4,800
2024-01-04	\$6,200
...	

Why it's time series:

- Each day's sales may depend on previous days
- Trends emerge over time (growing, declining)
- Seasonal patterns (weekends vs weekdays)

Example 2: Sensor Readings

Timestamp	Temperature
2024-01-08 00:00:00	68°F
2024-01-08 00:01:00	68.2°F
2024-01-08 00:02:00	68.5°F
...	

Why it's time series:

- Temperature changes gradually over time
- Diurnal patterns (day/night cycles)
- Trends (heating/cooling over hours)

Other Examples:

- **Stock prices:** Tick-by-tick or daily closing prices
- **Website traffic:** Visitors per hour/day

- **Energy consumption:** kWh per hour
- **Heart rate:** Beats per minute
- **Weather data:** Temperature, precipitation, wind speed
- **Economic indicators:** GDP, unemployment rate, inflation

NOT Time Series:

Height vs Weight of students - No time component, order doesn't matter **Customer attributes** - Age, gender, location (static) **Product catalog** - Price, category, rating (no temporal dependency)

Why Time Series Is Different:

Regular ML:

- Assumes data points are independent
- Can shuffle training data
- Cross-validation by random split

Time Series:

- Data points are dependent (autocorrelation)
- Cannot shuffle (destroys temporal structure)
- Must use time-based split (train on past, test on future)

Slide 15: Forecasting Fundamentals

Key Concepts:

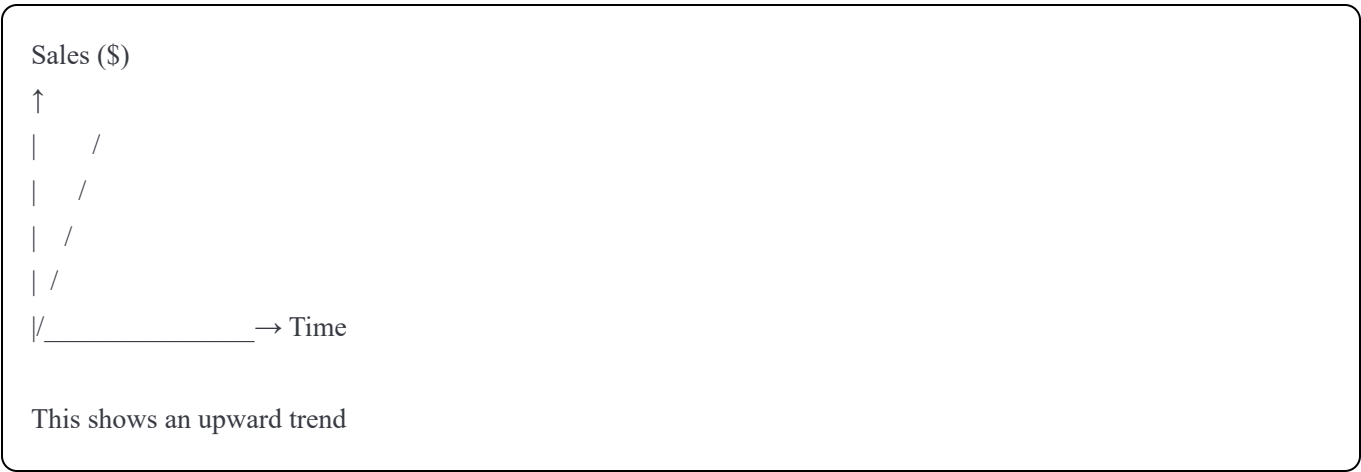
1. Trend

What it is: Long-term increase or decrease in the data

Examples:

- **Upward trend:** Company sales growing year-over-year
- **Downward trend:** Newspaper subscription declining
- **No trend:** Stationary data oscillating around constant mean

Visual:



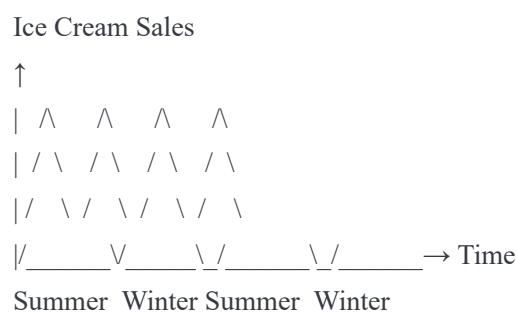
2. Seasonality

What it is: Regular, predictable patterns that repeat over fixed periods

Examples:

- **Daily:** Website traffic peaks during business hours, drops at night
- **Weekly:** Restaurants busy on weekends, slower on weekdays
- **Monthly:** Retail sales spike at month-end (payday effect)
- **Yearly:** Ice cream sales high in summer, low in winter

Visual:

**Seasonal Period:**

- Daily pattern: Period = 24 hours
- Weekly pattern: Period = 7 days
- Yearly pattern: Period = 12 months or 365 days

3. Noise

What it is: Random fluctuations that don't follow any pattern

Also called: Residuals, random error, white noise

Why it exists:

- Measurement errors
- Random events (weather, accidents)
- Unpredictable human behavior
- External factors not in the model

Example:

Expected sales: \$5,000/day (based on trend + seasonality)
Actual sales: \$5,200 (Monday), \$4,850 (Tuesday), \$5,100 (Wednesday)
Noise: +\$200, -\$150, +\$100

Decomposition Example:

$$\text{Observed Data} = \text{Trend} + \text{Seasonality} + \text{Noise}$$

Original Time Series: [5, 7, 6, 8, 9, 11, 10, 12, 13, 15, 14, 16]

Decomposed:

Trend: [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16] (growing)

Seasonality: [0, +2, 0, +2, 0, +2, 0, +2, 0, +2, 0, +2] (repeating pattern)

Noise: [0, -1, -1, -2, 0, -1, -1, -2, 0, -1, -1, -2] (random)

Common Forecasting Models:

1. ARIMA (AutoRegressive Integrated Moving Average)

What it is: Classical statistical model for time series forecasting

Components:

- **AR (AutoRegressive):** Use past values to predict future
 - "Today's value depends on yesterday's value"
 - Parameter p: How many past values to use
- **I (Integrated):** Make the series stationary through differencing
 - Remove trend by subtracting previous values
 - Parameter d: Order of differencing
- **MA (Moving Average):** Use past forecast errors
 - Learn from prediction mistakes
 - Parameter q: Number of past errors to use

ARIMA(p, d, q) Notation:

- ARIMA(1,1,1): 1 lag, 1 differencing, 1 error term
- ARIMA(2,1,2): 2 lags, 1 differencing, 2 error terms

When ARIMA Works Best:

- **Short-term forecasts:** Next few time steps
- **Stationary data:** Or can be made stationary
- **Linear relationships:** Past predicts future linearly
- **Sufficient data:** At least 50-100 observations

Example Use Case:

- Stock price prediction (next day)
- Daily sales forecasting
- Demand prediction

Limitations:

- Requires data to be stationary (or made stationary)
- Complex parameter tuning (p, d, q)
- Assumes linear relationships
- Not great with multiple seasonalities

2. Prophet (by Facebook/Meta)

What it is: Modern forecasting tool designed for business time series

Key Features:

- **Automatic seasonality detection:** Finds daily, weekly, yearly patterns
- **Holiday effects:** Account for special events
- **Robust to missing data:** Handles gaps gracefully
- **Trend changes:** Detects when trends shift
- **Easy to use:** Minimal parameter tuning

Model Components:

$y(t) = g(t) + s(t) + h(t) + \epsilon(t)$

Where:

$g(t)$ = Trend (piecewise linear or logistic growth)

$s(t)$ = Seasonality (Fourier series)

$h(t)$ = Holiday effects

$\epsilon(t)$ = Error/noise term

When Prophet Works Best:

- **Business forecasting:** Sales, revenue, user growth
- **Multiple seasonalities:** Daily + weekly + yearly patterns
- **Holiday effects matter:** Black Friday, Christmas, etc.
- **Missing data:** Irregular sampling intervals
- **Non-expert users:** No deep statistics knowledge required

Example Use Case:

- E-commerce daily sales forecasting
- Website traffic prediction
- Social media engagement forecasting

- Demand planning with promotions

Advantages:

- Interpretable: Can visualize trend, seasonality, holidays separately
- Fast: Fits in seconds
- Flexible: Easy to add custom seasonalities, regressors
- Robust: Handles outliers well

3. ML-Based Regressors

Approach: Treat time series as regression problem

How it works:

Create features from time series:

- Lag features: Value from t-1, t-2, t-7 days ago